Általános információk, a diplomaterv szerkezete

A diplomaterv szerkezete a BME Villamosmérnöki és Informatikai Karán:

1. Diplomaterv feladatkiírás
2. Címoldal
3. Tartalomjegyzék
4. A diplomatervező nyilatkozata az önálló munkáról és az elektronikus adatok kezeléséről
5. Tartalmi összefoglaló magyarul és angolul
6. Bevezetés: a feladat értelmezése, a tervezés célja, a feladat indokoltsága, a diplomaterv felépítésének rövid összefoglalása
7. A feladatkiírás pontosítása és részletes elemzése
8. Előzmények (irodalomkutatás, hasonló alkotások), az ezekből levonható következtetések
9. A tervezés részletes leírása, a döntési lehetőségek értékelése és a választott megoldások indoklása
10. A megtervezett műszaki alkotás értékelése, kritikai elemzése, továbbfejlesztési lehetőségek
11. Esetleges köszönetnyilvánítások
12. Részletes és pontos irodalomjegyzék
13. Függelék(ek)

Felhasználható a következő oldaltól kezdődő LaTeXdiplomatervsablon dokumentum tartalma.

A diplomaterv szabványos méretű A4-es lapokra kerüljön. Az oldalak tükörmargóval készüljenek (mindenhol 2,5 cm, baloldalon 1 cm-es kötéssel). Az alapértelmezett betűkészlet a 12 pontos Times New Roman, másfeles sorközzel, de ettől kismértékben el lehet térni, ill. más betűtípus használata is megengedett.

Minden oldalon – az első négy szerkezeti elem kivételével – szerepelnie kell az oldalszámnak.

A fejezeteket decimális beosztással kell ellátni. Az ábrákat a megfelelő helyre be kell illeszteni, fejezetenként decimális számmal és kifejező címmel kell ellátni. A fejezeteket decimális aláosztással számozzuk, maximálisan 3 aláosztás mélységben (pl. 2.3.4.1.). Az ábrákat, táblázatokat és képleteket célszerű fejezetenként külön számozni (pl. 2.4. ábra, 4.2. táblázat vagy képletnél (3.2)). A fejezetcímeket igazítsuk balra, a normál szövegnél viszont használjunk sorkiegyenlítést. Az ábrákat, táblázatokat és a hozzájuk tartozó címet igazítsuk középre. A cím a jelölt rész alatt helyezkedjen el.

A képeket lehetőleg rajzoló programmal készítsék el, az egyenleteket egyenlet-szerkesztő segítségével írják le (A LaTeX ehhez kézenfekvő megoldásokat nyújt).

Az irodalomjegyzék szövegközi hivatkozása történhet sorszámozva (ez a preferált megoldás) vagy a Harvard-rendszerben (a szerző és az évszám megadásával). A teljes lista névsor szerinti sorrendben a szöveg végén szerepeljen (sorszámozott irodalmi hivatkozások esetén hivatkozási sorrendben). A szakirodalmi források címeit azonban mindig az eredeti nyelven kell megadni, esetleg zárójelben a fordítással. A listában szereplő valamennyi publikációra hivatkozni kell a szövegben (a LaTeX-sablon a BibTeX segítségével mindezt automatikusan kezeli). Minden publikáció a szerzők után a következő adatok szerepelnek: folyóirat cikkeknél a pontos cím, a folyóirat címe, évfolyam, szám, oldalszám tól-ig. A folyóiratok címét csak akkor rövidítsük, ha azok nagyon közismertek vagy nagyon hosszúak. Internetes hivatkozások megadásakor fontos, hogy az elérési út előtt megadjuk az oldal tulajdonosát és tartalmát (mivel a link egy idő után akár elérhetetlenné is válhat), valamint az elérés időpontját.

Fontos:

- A szakdolgozatkészítő / diplomatervező nyilatkozata (a jelen sablonban szereplő szövegtartalommal) kötelező előírás, Karunkon ennek hiányában a szakdolgozat/diplomaterv nem bírálható és nem védhető!
- Mind a dolgozat, mind a melléklet maximálisan 15 MB méretű lehet!

Jó munkát, sikeres szakdolgozatkészítést, ill. diplomatervezést kívánunk!

FELADATKIÍRÁS

A feladatkiírást a tanszéki adminisztrációban lehet átvenni, és a leadott munkába eredeti, tanszéki pecséttel ellátott és a tanszékvezető által aláírt lapot kell belefűzni (ezen oldal *helyett*, ez az oldal csak útmutatás). Az elektronikusan feltöltött dolgozatban már nem kell beleszerkeszteni ezt a feladatkiírást.



Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Measurement and Information Systems

Elektronikus terelők

Bachelor's Thesis

Author
Jakab Gipsz

Advisor
dr. Egy Konzulens
Kettő Konzulens

November 3, 2026

Contents

Kivonat **i**

Abstract **ii**

1 Introduction **1**

2 Background **2**

2.1 Deployment strategies 2

2.2 Argo Rollouts 3

2.3 Metric-based analysis 3

3 Evaluation **4**

3.1 Research questions 4

3.2 Experimental setup 5

3.2.1 Environment and load 5

3.2.2 Fault injection 5

3.2.3 Detection mechanisms 5

3.2.4 Strategies 6

3.3 Trial automation 7

3.3.1 A trial as a sequence of commits 7

3.3.2 Observation 8

3.3.3 Measurements 8

3.4 Hypotheses 9

3.5 Results 10

3.5.1 Detection 10

3.5.2 Time to detect and to recover 11

3.5.3 User impact 13

3.5.4 Cost of a healthy release 14

3.6 The progress deadline that did not fire 14

3.6.1 Observation 15

3.6.2 Root cause . . . 15
3.6.3 Confirmation . . . 16
3.6.4 Relation to known issues and consequences . . . 17
3.7 Discussion . . . 17
3.8 Threats to validity . . . 18

4 LaTeX-eszközök 20
4.1 A szerkesztéshez használatos eszközök . . . 20
4.2 A dokumentum lefordítása Windows alatt . . . 21
4.3 Eszközök Linuxhoz . . . 22

5 A dolgozat formai kivitele 23
5.1 A dolgozat kimérete . . . 23
5.2 A dolgozat nyelve . . . 23
5.3 A dokumentum nyomdatechnikai kivitele . . . 23

6 A LaTeX-sablon használata 25
6.1 Címkék és hivatkozások . . . 25
6.2 Ábrák és táblázatok . . . 25
6.3 Felsorolások és listák . . . 27
6.4 Képletek . . . 28
6.5 Irodalmi hivatkozások . . . 29
6.6 A dolgozat szerkezete és a forrásfájlok . . . 32
6.7 Alapadatok megadása . . . 34
6.8 Új fejezet írása . . . 34
6.9 Definíciók, tételek, példák . . . 34

Acknowledgements 35

Bibliography 36

Appendix 39
A.1 Declaration on the Use of Generative Artificial Intelligence . . . 40
A.2 A TeXstudio felülete . . . 41
A.3 Válasz az „Élet, a világmindenség, meg minden” kérdésére . . . 42

STUDENT DECLARATION

I, *Jakab Gipsz*, the undersigned, hereby declare that the present Bachelor's Thesis work has been prepared by myself and without any unauthorized help or assistance. Only the specified sources (references, tools, etc.) were used. All parts taken from other sources word by word, or after rephrasing but with identical meaning, were unambiguously identified with explicit reference to the sources utilized. In the Appendix, I have clearly indicated whether I used artificial intelligence tools in preparing the thesis; if so, I have specified the manner and extent of their use in the table. I acknowledge that I bear full responsibility for any content generated by artificial intelligence – regardless of the extent of its use.

I agree that the basic information of my present work (author(s), title, abstracts in English and in a second language, year of preparation, name of advisor(s)) will be made publicly available on the internet by BME VIK, and the full text of the work will be available to persons registered in the BME Directory. I declare that the submitted hardcopy of the thesis work and its electronic version are identical.

Full text of thesis works classified upon the decision of the Dean will be published after a period of three years.

Budapest, November 3, 2026

Jakab Gipsz
student

Kivonat

Jelen dokumentum egy diplomaterv sablon, amely formai keretet ad a BME Villamosmérnöki és Informatikai Karán végző hallgatók által elkészítendő szakdolgozatnak és diplomatervnek. A sablon használata opcionális. Ez a sablon LaTeX alapú, a *TeXLive* TeX-implementációval és a PDF-LaTeX fordítóval működőképes.

Abstract

This document is a LaTeX-based skeleton for BSc/MSc theses of students at the Electrical Engineering and Informatics Faculty, Budapest University of Technology and Economics. The usage of this skeleton is optional. It has been tested with the *TeXLive* TeX implementation, and it requires the PDF-LaTeX compiler.

Chapter 1

Introduction

A bevezető tartalmazza a diplomaterv-kiírás elemzését, történelmi előzményeit, a feladat indokoltságát (a motiváció leírását), az eddigi megoldásokat, és ennek tükrében a hallgató megoldásának összefoglalását.

A bevezető szokás szerint a diplomaterv felépítésével záródik, azaz annak rövid leírásával, hogy melyik fejezet mivel foglalkozik.

Chapter 2

Background

This chapter introduces the deployment strategies and tools the rest of the thesis builds on. Section 2.1 compares the common ways of releasing a new version, Section 2.2 describes Argo Rollouts, the controller that performs the canary releases, and Section 2.3 explains how metrics collected by Istio and Prometheus decide whether a release is rolled back.

2.1 Deployment strategies

Continuous delivery aims to keep software in a releasable state and to make each release a small, routine event [1]. Small releases lower the risk of each change, but they do not remove it: a version that passes every test can still fail under production traffic. The deployment strategy determines how many users such a faulty version reaches before it is detected.

The built-in Kubernetes *Deployment* object uses a rolling update by default: it replaces old pods with new ones in batches, limited by the `maxSurge` and `maxUnavailable` parameters [2]. A pod only receives traffic once its readiness probe succeeds, so a version that does not start is stopped early. A version that starts but returns errors or responds slowly, however, is rolled out to every pod. If the update does not finish within `progressDeadlineSeconds`, Kubernetes marks it as failed, but it does not roll it back automatically.

Progressive delivery strategies expose the new version gradually and check its health at each step. Table 2.1 summarises the strategies relevant to this work.

Strategy	Exposure of the new version	Automatic rollback
Recreate	all users at once, with downtime	no
Rolling update	all users, pod by pod	no
Blue-green	all users after a single switch	on failed checks before the switch
Canary	a growing share of users	on failed metric checks at any step

Table 2.1: Comparison of deployment strategies.

A canary release sends a small share of traffic to the new version, compares its behaviour with the stable version, and only increases the share while the comparison is favourable [3]. Its weakness is that some real users are still served by the faulty version during the first step. Traffic *mirroring* avoids this: the service mesh copies requests to the canary and

discards its responses, so the canary can be measured on real requests before any user depends on it.

2.2 Argo Rollouts

Argo Rollouts is a Kubernetes controller that replaces the Deployment with a *Rollout* custom resource supporting canary and blue-green strategies [4]. A canary Rollout lists its steps explicitly, for example `setWeight` to send a percentage of traffic to the canary, `pause` to wait, and `setMirrorRoute` to mirror requests. The controller does not route traffic itself; it rewrites the routing rules of a traffic provider such as Istio.

The health of a release is defined in an *AnalysisTemplate*. It contains one or more metrics, each with a query, an evaluation interval, a `successCondition` and a `failureLimit`. During a release the controller creates an *AnalysisRun* from the template and evaluates the metrics periodically. When a metric fails more times than its `failureLimit` allows, the Rollout is aborted: all traffic returns to the stable version and the canary is scaled down.

2.3 Metric-based analysis

Istio is a service mesh: it places an Envoy proxy next to every application container, and these proxies handle all traffic between services [5]. Because each request passes through a proxy, Istio can split or mirror traffic according to a *VirtualService*, and it records standard metrics for each request, such as `istio_requests_total` and `istio_request_duration_milliseconds`, without changing the application.

Prometheus collects these metrics and evaluates queries written in PromQL [6]. Listing 2.1 shows the success-rate metric of the AnalysisTemplate used in this work. The query divides the rate of non-5xx responses of the canary by the rate of all its responses over the last minute, and the metric fails if the result drops below 95%.

```
- name: success-rate
  interval: 30s
  initialDelay: 30s
  failureLimit: 1
  successCondition: result[0] >= 0.95
  provider:
    prometheus:
      address: http://prometheus-server.monitoring.svc.cluster.local
      query: |
        (sum(rate(istio_requests_total{reporter="source",
          destination_service_name="demo-canary",
          response_code!~"5.*"}[1m])) or vector(0))
        /
        sum(rate(istio_requests_total{reporter="source",
          destination_service_name="demo-canary"}[1m]))
```

Listing 2.1: Success-rate metric of the canary-health AnalysisTemplate.

With `failureLimit: 1`, one failed measurement is tolerated and the second one aborts the release. This prevents a single noisy measurement from rolling back a healthy version, while a genuinely faulty one is still detected within about one minute.

Chapter 3

Evaluation

This chapter evaluates the automated rollback platform experimentally. Three detection mechanisms (traffic mirroring, a baseline experiment and a metric-checked canary) are combined in every possible way, a blue-green release is added for comparison, and each of the resulting nine strategies is released with eight versions of the demo application, one healthy and seven faulty. The 72 releases were run, observed and measured by an automated trial runner without manual intervention.

Section 3.1 states the research questions. Section 3.2 describes the environment, the faults, the mechanisms and the strategies, and Section 3.3 the trial runner and how it measures a release. Section 3.4 gives the hypotheses, which were written down before the first run. Section 3.5 presents the results, and Section 3.6 analyses the only result that contradicted the hypotheses: a progress deadline that never fired. Section 3.7 answers the research questions, and Section 3.8 discusses the threats to validity.

3.1 Research questions

The experiments answer five questions.

RQ1 Which detection mechanism catches which kind of fault, and how fast?

RQ2 What does each strategy cost, in users exposed to a faulty version and in the time and resources of a healthy release?

RQ3 How do absolute checks (fixed thresholds on the new version alone) compare with relative checks (the new version compared with a fresh copy of the old one)?

RQ4 Do combined mechanisms protect better than single ones, and which combinations only add time?

RQ5 What changes when the gradual canary is replaced by an all-at-once blue-green switch?

The scope is limited to faults *in the release itself*: the new version is faulty, while the cluster, the network and the dependencies are healthy. Environment faults, such as a node under CPU pressure, are left for future work (Section 3.8).

3.2 Experimental setup

3.2.1 Environment and load

All runs used the development environment of the platform: a single-node k3s cluster with Istio as the service mesh, Prometheus as the metric store, Argo Rollouts v1.10.1 as the release controller and Argo CD as the GitOps agent [7]. Every change to an application, including the image tag that triggers a release, is a commit in the platform's Git repository, which Argo CD applies to the cluster.

The demo application is a small HTTP service with one replica per version. A load generator inside the cluster sends a request to `GET /api/work` every 0.2 s through the Istio ingress gateway, about 5 requests per second in total, at the same rate in every run. Prometheus scrapes the Envoy sidecars every 10 s. Because a single request is usually not measured on its own, every time below is only accurate to about 10–20 s.

3.2.2 Fault injection

Each fault is a separate image of the demo application, built by the CI pipeline from the same source commit with a different build setting (Table 3.1). A fault is therefore part of the version being released, and it is present from the first request on, exactly as a bug shipped in a release would be. The faults were chosen to cover different kinds of failure: loud and fast (F1, F3), below a threshold (F2), noisy (F4), slow-burning (F5), failing at start-up (F6) and semantic, invisible to request metrics (F8). F0 is the healthy control, used to detect false alarms and to measure the cost of a release that should simply succeed.

ID	Fault	Build setting	Kind
F0	healthy version	none	control
F1	30 % of requests return HTTP 500	`ERROR_RATE=30`	loud, fast
F2	2 % of requests return HTTP 500	`ERROR_RATE=2`	below every threshold
F3	every response delayed by 800 ms	`LATENCY_MS=800`	loud, fast
F4	random delay of 0–1500 ms	`LATENCY_JITTER_MS=1500`	noisy tail latency
F5	memory grows by 20 MiB per minute	`LEAK_MB_PER_MIN=20`	slow-burn
F6	readiness probe always fails	`FAIL_READINESS=true`	start-up
F8	wrong result with HTTP 200	`WRONG_ANSWER=true`	semantic

Table 3.1: Injected faults. Each is a separate image built from the same source commit. (F7, an environment fault, was dropped from the scope; its identifier was kept free.)

3.2.3 Detection mechanisms

Three mechanisms can protect a canary release, and each can be switched on or off independently. When several are on, they always run in the same order, mirroring first, then the experiment, then the canary steps.

M – traffic mirroring. The new version is scaled to one pod while it receives no user traffic, and Istio copies every `GET /api/work` request to it [5]. The responses of the copy are discarded, so no user ever sees them. During mirroring, the AnalysisTemplate `mirror-health` takes three measurements 20 s apart, after a 30 s initial delay. It measures the success rate (at least 95 %, no failed measurement allowed) and the 95th percentile

latency (at most 300 ms, one failure allowed) of the requests the new pods received. It has no memory check, because mirroring lasts only about 70 s.

E – baseline experiment. An Argo Rollouts *experiment* step starts two fresh pods side by side for three minutes, a *baseline* running the stable version and a *canary* running the new one, and sends 20 % of the users to each [8]. The AnalysisTemplate `canary-vs-baseline` compares the two every 20 s: the canary's 5xx ratio may exceed the baseline's by at most 5 percentage points, its p95 latency by at most 100 ms, and its memory growth by at most 0.05 MiB/s. Because both pods start at the same moment, effects of start-up, such as the initial memory allocation, cancel out in the difference. This makes E the only *relative* check of the platform.

C – canary steps. The canary receives 10 % of the users for one minute, then 50 % for one minute, before being promoted. A background analysis, `canary-health`, runs during these steps and checks the success rate (at least 95 %) and the p95 latency (at most 300 ms) every 30 s, with one failure allowed, and the memory growth (at most 0.05 MiB/s) every 30 s after a 90 s delay, with no failure allowed. These are *absolute* thresholds, applied to the new version on its own.

BG – blue-green with a smoke test. The blue-green variant of the demo application uses a separate Rollout with an active and a preview Service [9]. The new version first starts behind the preview Service, which receives no user traffic. A pre-promotion analysis then runs a Kubernetes Job [10], which sends 50 requests to the preview Service and fails if any of them returns an error or an incorrect result. Only then are all users switched to the new version at once. A post-promotion analysis, `bg-post-promotion`, then checks the success rate, the p95 latency and the memory growth of the now active version for about two and a half minutes, and aborts the release (switching the users back) if they fail. The Job runs inside the mesh under strict mutual TLS, which requires the native sidecar mode of Kubernetes, so that the Envoy proxy starts before the test and stops after it [11].

The progress deadline. Independently of the strategy, every Rollout sets `progressDeadlineSeconds: 120` and `progressDeadlineAbort: true` [12]. If a release makes no progress for 120 s, for example because its pods never become Ready, the controller should abort it. This is a property of the Rollout, not a step, so it is part of every strategy including the control.

3.2.4 Strategies

The three canary mechanisms were combined in a full factorial design [13]: each of M, E and C is either on or off, which gives the eight strategies S0–S7 of Table 3.2, plus the blue-green strategy BG. The single-mechanism strategies S1–S3 show what each mechanism catches on its own, while S4–S7 show whether a combination catches more, or only takes longer.

A release must end at 100 %: when the last step finishes, the Rollout promotes the new version. A strategy without C therefore sends all users to the new version right after its last check, and a fault that this check missed reaches everyone. This is the honest standalone behaviour of the mechanism, and the reason why S2 and S3 are not merely

shorter versions of S7. S0, with no steps at all, is the control: it releases the new version to all users at once, protected only by the progress deadline.

ID	M	E	C	Meaning
S0				control: no progressive delivery, only the progress deadline
S1			✓	canary alone
S2	✓			mirroring alone
S3		✓		experiment alone
S4	✓		✓	mirroring, then canary
S5		✓	✓	experiment, then canary
S6	✓	✓		mirroring, then experiment
S7	✓	✓	✓	all three (the platform's default configuration)
BG				blue-green: preview, smoke Job, switch, post-promotion analysis

Table 3.2: The strategies under test. M: mirroring, E: baseline experiment, C: canary steps.

The parameters of each mechanism (durations, weights, intervals and thresholds) are the same in every strategy, so any difference between two strategies comes only from the mechanisms that differ between them. The strategies are implemented as Kustomize components [14], one per strategy, each of which replaces the steps of the default Rollout (S7). The component `s7-all` is empty, so the default configuration is tested exactly as it is deployed. A script generates the components from shared blocks, which guarantees that the M, E and C steps are byte-for-byte identical wherever they appear. Switching the strategy changes a single line of the development overlay.

3.3 Trial automation

Running 72 releases by hand, with a clock and a dashboard, would be slow and, worse, inconsistent. The trial runner, a Python program using only the standard library, performs every run in the same way and records what happened in enough detail to recompute every measurement later.

3.3.1 A trial as a sequence of commits

Because Argo CD reverts any change made to the cluster directly, the runner changes the platform the same way a developer would: through Git. A trial consists of the following steps.

1. **Preconditions.** The other application must not be releasing, and the application under test must be Healthy on its reset tag, a healthy image. Otherwise the runner resets it first.
2. **Strategy.** If the strategy differs from the current one, the runner edits the component line of the overlay, commits, pushes, asks Argo CD to refresh, and waits until Argo CD reports this commit as synced.
3. **Trigger.** Once the observer has recorded the initial state, the runner commits the image tag of the fault under test. This commit starts the release.
4. **Observation.** The runner follows the release until it is promoted, or until it is aborted and no user traffic reaches the new version for 10 s. If neither happens

within 20 minutes (4 minutes for F6), it records a timeout and aborts the release itself.

5. **Measurement.** The timeline and Prometheus are turned into measurements, which are saved as one row of a CSV file and, together with the raw timeline, as a JSON record.

6. **Reset.** The runner commits the reset tag. After an abort this is instant, since the stable version is already running. After a promotion the faulty version has become stable, so the reset is itself a release; it runs under S0 to take seconds instead of minutes. A 60 s pause then separates the trial from the next one, so that the one-minute rate windows of the queries do not overlap.

Every trial is thus traceable to the commits that defined its strategy and triggered its release. The runner groups the trials by strategy to minimise switches, and skips trials that already have a result, so an interrupted matrix can be resumed.

3.3.2 Observation

Every two seconds, the runner reads the Rollout, its AnalysisRuns and Experiments, the VirtualService, the Services and the pods of the application in a single `kubectl` call, and reduces them to a small *snapshot*: the phase, step and abort flag of the Rollout, the current and stable pod-template hashes, the route weights and the mirror target, the phase and failure count of each metric of the new version, and the pods. A snapshot is stored only when it differs from the previous one, so the *timeline* of a trial is the list of its state changes, each with a timestamp.

Where users go is read from the routing objects themselves, not from the Rollout's status, because these are what actually route the traffic. For a canary, this is the weight of the canary destinations in the VirtualService; for blue-green, the pod-template hash selected by the active Service. Once the new version is promoted, the stable Service points to it, so it serves 100 % of the users.

3.3.3 Measurements

Table 3.3 defines the measurements. Times come from the timeline. The moment of detection is the `finishedAt` time of the first failing measurement, recorded by the controller in cluster time and converted with the clock offset between the cluster and the runner, which the runner measures at the start of each trial.

Request counts come from the Istio standard metrics `istio_requests_total` and `istio_request_duration_milliseconds` [15], reported by the sidecars of the new version and selected by its pod-template hash, so that every route to it is counted, including the stable Service after a promotion. The experiment's canary pod has its own hash, so it is counted through its own Service. Mirrored copies also come from the gateway, so the user window starts only when mirroring has ended. Mirrored copies and smoke-test requests are counted separately. The threshold for a slow request is 500 ms rather than the 300 ms of the latency checks, because Istio's histogram has no bucket between 250 and 500 ms [16].

Exact counting. The usual way to count requests in a window, PromQL's `increase()`, extrapolates the counter to the edges of the window [6]. For short bursts this overcounts

Measurement	Definition
outcome	*promoted*, *aborted*, or *timeout* if the release did not end by itself
caught by	the check whose failing measurement finished first, or the progress deadline
TTD	time to detect: from the first request of any kind to the new version (a mirrored copy, a user request or a smoke-test request) until detection
TTR	time to recover: from detection until no user and no mirrored request reaches the new version
max. exposure	the highest share of users routed to the new version
user requests	requests the ingress gateway sent to the new version, counted at the new pods' sidecars; of these, *affected* are those answered with a 5xx code or slower than 500 ms
duration	for a promoted release: from the start until the promotion
pod-minutes	the summed lifetime of the pods created during the trial

Table 3.3: Measurements recorded for every trial.

considerably: in a validation run, it reported 63 smoke-test requests where the Job's log showed exactly 50. The runner therefore takes the last value of each counter series up to the end of the window and subtracts its last value before the start, treating a series that did not exist yet as zero. With this method the same run gives exactly 50. The windows end 30 s after the event that closes them, to include the final scrape.

Validation. Before the matrix, the runner was validated on a recorded blue-green release (promoted, 50 smoke requests, no errors), on a synthetic aborted canary with mirroring, and on a live faulty blue-green release. These runs revealed two defects, which were fixed before the matrix: snapshots taken before the release started still showed the analysis runs of the previous release, and the recorded Git commit did not yet include the trigger. The matrix itself revealed four more, all in the measurement code and none in the observation, so the stored timelines were re-measured afterwards: promoted releases were reported with 0 % exposure, their user requests were undercounted, progress-deadline aborts were labelled with the raw controller message, and pods that appeared in a single snapshot were counted with zero lifetime.

3.4 Hypotheses

Before the first run, the expected result of every cell was written down from the configuration alone (Table 3.4). With combined mechanisms, the first one in the order M, E, C that catches a fault should decide the outcome, and the later ones should never run. Some predictions follow directly from the design:

- F2 should pass every metric check, since 2 % errors is below both the 95 % success threshold and the 5-point delta. Blue-green should catch it only by chance: 50 requests all succeed with probability $0.98^{50} \approx 36\,\%$, so the Job fails in about 64 % of the runs.
- F8 returns correct HTTP status codes with a wrong body, so only the smoke Job, which checks the body, should catch it.
- Mirroring has no memory check and lasts about 70 s, so it should let F5 through to all users (S2), or pass it on to the next mechanism (S4, S6, S7).

- The canary's memory check starts after 90 s, when the canary is already at 50 %, so C should catch F5 at 50 % exposure.
- Placing E before C should make the loud faults *worse* than C alone, because E exposes 20 % of the users while the canary's first step exposes only 10 %.
- F6 never becomes Ready and therefore never receives any traffic, so in every strategy it should be stopped by the progress deadline after about 120 s, with no user exposed.

	F0	F1	F2	F3	F4	F5	F6	F8
S0	pass	×	×	×	×	×	DL · 0 %	×
S1 C	pass	C · 10 %	×	C · 10 %	C · 10 %	C · 50 %	DL · 0 %	×
S2 M	pass	M · 0 %	×	M · 0 %	M · 0 %	×	DL · 0 %	×
S3 E	pass	E · 20 %	×	E · 20 %	E · 20 %	E · 20 %	DL · 0 %	×
S4 M+C	pass	M · 0 %	×	M · 0 %	M · 0 %	C · 50 %	DL · 0 %	×
S5 E+C	pass	E · 20 %	×	E · 20 %	E · 20 %	E · 20 %	DL · 0 %	×
S6 M+E	pass	M · 0 %	×	M · 0 %	M · 0 %	E · 20 %	DL · 0 %	×
S7 M+E+C	pass	M · 0 %	×	M · 0 %	M · 0 %	E · 20 %	DL · 0 %	×
BG	pass	Job · 0 %	Job (64 %)	post · 100 %	post · 100 %	post · 100 %	DL · 0 %	Job · 0 %

Table 3.4: Hypotheses, written before the first run: the check expected to stop the release, and the highest share of users exposed. ×: not caught, the faulty version reaches all users. DL: progress deadline. Job: pre-promotion smoke test. post: post-promotion analysis. pass: the healthy version was promoted, as it should be.

3.5 Results

The 72 trials ran unattended on 1 November 2026 between 13:15 and 17:35, in 4 hours and 20 minutes, 3.6 minutes per trial on average including the resets and pauses. Every trial completed; none had to be repeated.

3.5.1 Detection

Table 3.5 shows what stopped each release and the highest share of users exposed to it. **70 of the 72 cells match the hypotheses.** The two exceptions are S6 and S7 with F6, where the progress deadline did not fire, and the release stayed at its first step until the runner aborted it after four minutes. Section 3.6 analyses these two cells. In both, no user was exposed, since the new version never became Ready.

Some cells match the hypotheses in a way worth noting:

- **No false alarms.** All nine healthy releases (F0) were promoted.
- **F6 under S3 and S5** was stopped by the deadline of the experiment, not of the Rollout. These strategies start with the experiment step, whose canary template has its own progress deadline; it expired after 120 s and failed the experiment, which aborted the Rollout. The outcome is the same as predicted.
- **F2 under BG** was caught: one of the 50 smoke requests failed. This is the more likely of the two possible outcomes (about 64 %). A single run cannot show the rate.

	F0	F1	F2	F3	F4	F5	F6	F8
S0	pass	×	×	×	×	×	DL · 0 %	×
S1 C	pass	C · 10 %	×	C · 10 %	C · 10 %	C · 50 %	DL · 0 %	×
S2 M	pass	M · 0 %	×	M · 0 %	M · 0 %	×	DL · 0 %	×
S3 E	pass	E · 20 %	×	E · 20 %	E · 20 %	E · 20 %	DL_E · 0 %	×
S4 M+C	pass	M · 0 %	×	M · 0 %	M · 0 %	C · 50 %	DL · 0 %	×
S5 E+C	pass	E · 20 %	×	E · 20 %	E · 20 %	E · 20 %	DL_E · 0 %	×
S6 M+E	pass	M · 0 %	×	M · 0 %	M · 0 %	E · 20 %	**hang** · 0 %	×
S7 M+E+C	pass	M · 0 %	×	M · 0 %	M · 0 %	E · 20 %	**hang** · 0 %	×
BG	pass	Job · 0 %	Job · 0 %	post · 100 %	post · 100 %	post · 100 %	DL · 0 %	Job · 0 %

Table 3.5: Measured results: the check that stopped the release, and the highest share of users exposed. DL_E: the experiment's own progress deadline. hang: no check stopped the release. Other symbols as in Table 3.4.

- **F5 under S4, S6 and S7** passed mirroring, as predicted, and was caught by the next mechanism: by the canary at 50 % in S4, and by the experiment at 20 % in S6 and S7.
- **F8** was caught only by the smoke Job's body check; every canary strategy promoted it.

3.5.2 Time to detect and to recover

Table 3.6 lists the time to detect. Each mechanism has a characteristic detection time, set mainly by its initial delay, its interval and the number of failures it tolerates, and much less by the fault:

- Mirroring catches F1 in 27–29 s: its success-rate metric tolerates no failure, so the first measurement after the 30 s delay decides. It catches latency in 47–49 s, because the latency metric tolerates one failure, so the second measurement decides.
- The experiment catches errors and latency in 47–51 s (one failure allowed, 20 s interval) and the leak in 57–61 s.
- The canary catches errors and latency in 57–59 s (one failure allowed, 30 s interval) and the leak in 87 s.
- The smoke Job catches F1, F2 and F8 in 15–17 s, the time needed to start the Job and send 50 requests.
- The post-promotion analysis catches F3–F5 in 82–111 s, all of it with every user on the faulty version.

The run-to-run differences between faults caught by the same check were within one measurement interval. Noisy latency (F4) was caught as quickly as constant latency (F3), contrary to the expectation that it would be slower and more variable: a random delay of up to 1500 ms puts the p95 at about 1400 ms, far above the 300 ms threshold.

For F5 under S4, S6 and S7, the time to detect (139–170 s) is long only because it counts from the first mirrored request: the leak went unnoticed during mirroring and was caught by the next mechanism. Measured from the start of the experiment or the canary step, it is the same as under S3 or S1.

Figure 3.1 plots the detected runs by time to detect and exposure. They fall into clear groups, one per mechanism: mirroring and the smoke Job at 0 %, the experiment at 20 %,

	F1	F2	F3	F4	F5	F6	F8
S0	–	–	–	–	–	(121)	–
S1 C	57	–	58	59	87	(121)	–
S2 M	28	–	48	49	–	(119)	–
S3 E	47	–	47	50	57	(122)	–
S4 M+C	27	–	48	49	170	(119)	–
S5 E+C	49	–	51	47	61	(120)	–
S6 M+E	27	–	48	48	139	hang	–
S7 M+E+C	29	–	48	47	144	hang	–
BG	16	16	111	100	82	(121)	15

Table 3.6: Time to detect, in seconds, from the first request of any kind to the new version. –: not detected (promoted). F6 never receives a request, so its column (in parentheses) is measured from the start of the release.

the canary at 10 % (and at 50 % for the leak), and the post-promotion analysis at 100 %. Only mirroring and the smoke Job detect a fault without exposing any user, and neither of them caught the leak.

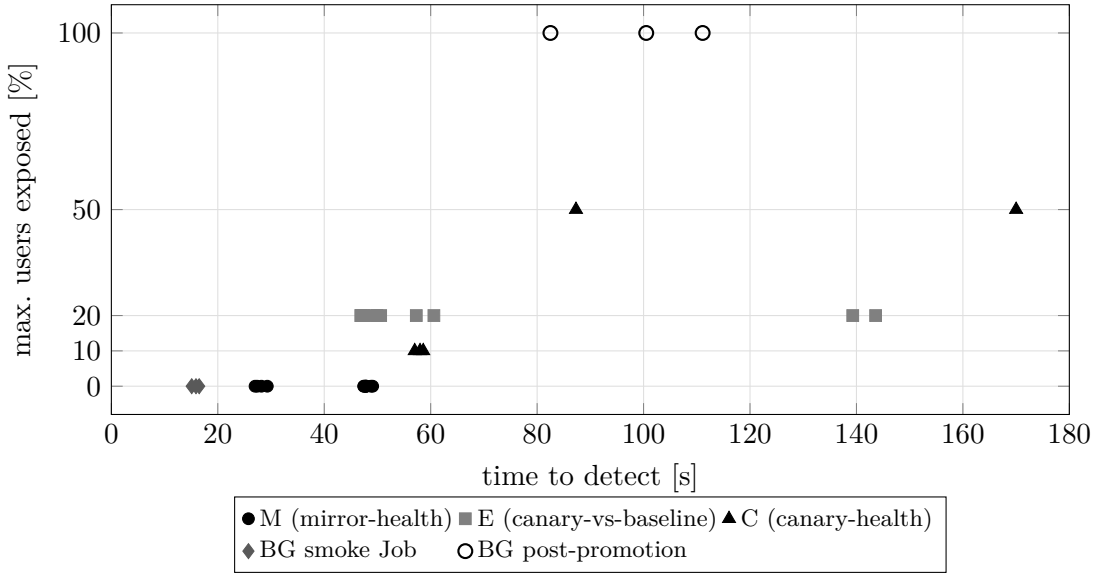


Figure 3.1: Detected faults F1–F5 and F8 by time to detect and by the highest share of users exposed, one point per run, marked by the check that caught the fault. The points at 139–170 s are F5 under S4, S6 and S7, counted from the first mirrored request.

Recovery was fast in every case: once a check failed, the controller moved all traffic back to the stable version within 0.6–3.2 s (median 1.8 s, over the 33 runs caught by a metric or the smoke Job). This time is dominated by the 2 s polling interval of the runner, so the true recovery time is likely shorter. Rolling back is effectively instant, because the stable version keeps running throughout a canary release; what decides the damage is how long detection takes and how many users are exposed until then.

3.5.3 User impact

Table 3.7 shows how many user requests reached the faulty version, and how many of them were affected (an error or slower than 500 ms). Strategies that start with mirroring kept every loud fault away from the users entirely: under S2, S4, S6 and S7, F1, F3 and F4 reached no user. The mirrored copies show what users would have experienced: of the 120–131 copies received during F1, 36–45 failed, about the configured 30 %.

	F1	F2	F3	F4	F5	F6	F8
S0	36/10[P]	32/2	6/6[P]	8/4[P]	26/0[P]	0	35/0
S1 C	21/6	189/5	23/23	17/12	90/0	0	188/0
S2 M	0	38/1	0	0	55/0[P]	0	75/0
S3 E	34/16	207/3	26/26	24/17	38/0	0	197/0
S4 M+C	0	204/3	0	0	83/0	0	244/0
S5 E+C	42/11	332/1	27/27	30/16	54/0	0	340/0
S6 M+E	0	207/1	0	0	73/0	0	197/0
S7 M+E+C	0	332/8	0	0	79/0	0	331/0
BG	0	0	48/48	51/31	249/0	0	0

Table 3.7: User requests served by the faulty version / of these, answered with a 5xx code or slower than 500 ms. [P]: the version was promoted (in the F2 and F8 columns, every strategy except BG promoted it); the count ends 30 s after the promotion, after which the fault stays with every user until someone notices it.

Three results deserve attention.

The experiment exposes more users than the canary. For the loud faults, S3 and S5 exposed more users than S1: F1 reached 34 and 42 users under S3 and S5 (16 and 11 of them failed), but only 21 under S1 (6 failed). F3 affected 26–27 users instead of 23. This confirms the prediction: the experiment sends 20 % of the users to the new version from the start, twice the canary's first step, and it detects in about the same time. Placed before a small canary step, the experiment makes the release less safe for exactly the faults that are easiest to catch.

The experiment is better for slow faults. For the leak (F5) the order is reversed: the canary caught it only at 50 %, after 90 user requests, while the experiment caught it at 20 % after 38 (S3) or 54 (S5). The relative memory check needs no long initial delay, because the baseline makes the same start-up allocation as the canary and the difference cancels it out. The canary's absolute check has to wait 90 s for the start-up allocation to settle, otherwise it would abort healthy releases; by then the canary has reached 50 %.

Blue-green trades early safety for late exposure. The smoke Job caught F1, F2 and F8 before any user was switched, but every fault it does not test for reached all users: F3 slowed down all 48 user requests during the post-promotion analysis, and F5 served 249 user requests before its memory growth was detected. The leak caused no visible errors in this short window, but it would eventually have exhausted the pod's memory.

3.5.4 Cost of a healthy release

Table 3.8 and Figure 3.2 show the cost of the healthy release, which every strategy pays on every release, faulty or not. The cost grows with every mechanism: from 11 s without protection to 6.8 minutes with all three, and from 0.2 to 12.7 pod-minutes. The experiment is the most expensive mechanism: it adds 3.1–3.4 minutes and, because it runs two additional pods, 6.6–8.9 pod-minutes to each release (S0 to S3, S1 to S5, S4 to S7). Mirroring adds 1.4–1.6 minutes and doubles the requests the new version handles during that time (288–334 mirrored copies), which would double the load on its downstream dependencies in a real system.

	duration [s]	duration [min]	pod-minutes	mirrored or smoke requests
S0	11	0.2	0.2	–
S1 C	128	2.1	2.1	–
S2 M	97	1.6	1.9	334 mirrored
S3 E	201	3.4	6.8	–
S4 M+C	221	3.7	4.3	294 mirrored
S5 E+C	332	5.5	11.0	–
S6 M+E	299	5.0	9.4	290 mirrored
S7 M+E+C	409	6.8	12.7	288 mirrored
BG	159	2.7	2.7	50 smoke

Table 3.8: Cost of a healthy release (F0): time from start to promotion, and the summed lifetime of the pods created during the trial.

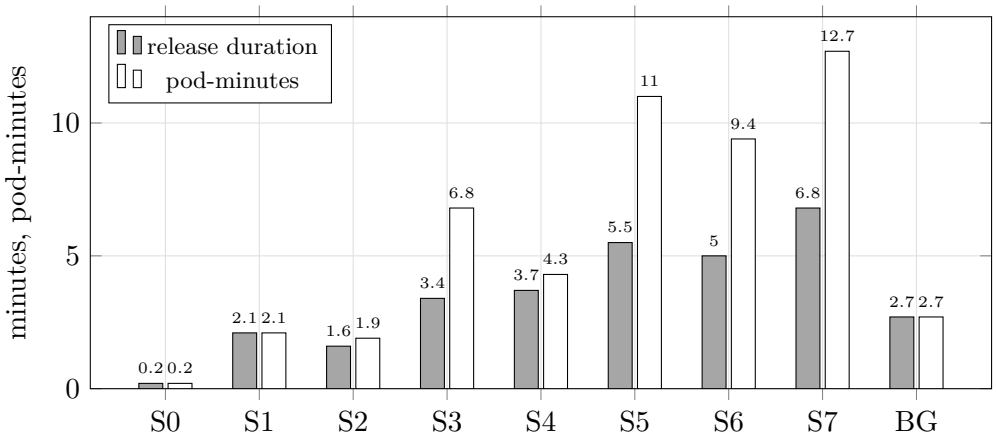


Figure 3.2: Duration and pod-minutes of a healthy release per strategy.

The measured durations were close to those estimated in the plan (S1 2.1 minutes against about 2.5, S7 6.8 against about 7.5).

3.6 The progress deadline that did not fire

The progress deadline is the platform's safety net for a version that never starts. It worked in S0–S5 and in BG, but in S6 and S7 the F6 release stayed at step 0 with its pod not Ready, the Rollout reported *Progressing* indefinitely, and nothing happened until the runner aborted it after four minutes. No user was harmed, because a pod that is not Ready receives no traffic. But a release that never ends blocks every later release of the application, and nobody is told: the Rollout does not even become *Degraded*.

3.6.1 Observation

The two affected strategies have both mirroring and the experiment; the strategies with only one of them (S2, S3, S4, S5) were not affected. Yet in S6 and S7 the F6 release never got beyond the first step, `setCanaryScale`, so the steps that differ between the strategies had not even started. The cause therefore had to lie in the state the cluster was in *before* the release, not in the release itself.

The state of the ReplicaSets after the matrix gave the decisive clue. The ReplicaSet of the F5 image, scaled to zero replicas, still carried a `scale-down-deadline` annotation, with a time 30 s after the abort of the S7–F5 trial, which ran immediately before S7–F6. The controller adds this annotation when it aborts a release, to keep the canary running for `abortScaleDownDelaySeconds` (30 s by default) before scaling it down.

3.6.2 Root cause

The source code of the controller [17] explains how such an annotation disables the deadline. The deadline is evaluated in `rollout/sync.go`, but only when the following guard is true:

```
case !isIndefiniteStep(c.rollout) &&
     !isWaitingForReplicaSetScaleDown(c.rollout, c.newRS, c.stableRS, c.allRSs) &&
     conditions.RolloutTimedOut(c.rollout, newStatus):
```

Listing 3.1: The guard of the progress-deadline check in rollout/sync.go (Argo Rollouts v1.10.1).

The function `isWaitingForReplicaSetScaleDown` returns true if *any* ReplicaSet of the Rollout, other than the new and the stable one, has a `scale-down-deadline` annotation. It only checks that the annotation exists, not whether the deadline has passed or whether the ReplicaSet has any replicas left. The intent is reasonable: while an old version is kept running for a delay, the release is not stuck. But a stale annotation keeps the guard false forever, so the deadline is never evaluated.

Three behaviours of the controller combine to leave such a stale annotation behind:

1. **The annotation is added even to an empty ReplicaSet.** When a release is aborted, `reconcileNewReplicaSet` (`rollout/replicaset.go`) annotates the canary ReplicaSet as soon as the stable version is fully available. It does not check whether the canary has any replicas to keep.

2. **Scaling down during an abort keeps the annotation.** `scaleReplicaSet` removes the annotation when it scales a ReplicaSet to zero, but not while the Rollout is aborted (`shouldDelayScaleDownOnAbort` is true).

3. **Empty old ReplicaSets are skipped.** After the reset, the aborted ReplicaSet is an old one. `scaleDownOldReplicaSetsForCanary` (`rollout/canary.go`) is where old ReplicaSets lose their annotation, by being scaled to zero; but it skips any ReplicaSet that is already at zero replicas.

Whether a release leaves the annotation behind therefore depends on whether the canary ReplicaSet still has replicas when the release is aborted. In S6 and S7, F5 was caught by the experiment. During the experiment step the Rollout's own canary ReplicaSet is at zero, since the canary weight is zero and the experiment runs its own pods. The abort annotated this empty ReplicaSet. The reset 13–14 s later cleared the abort, and from then

on the ReplicaSet was an old, empty one, which nothing ever touched again. In S1 and S4, by contrast, F5 was caught at 50 % with the canary pod running. After the reset this ReplicaSet was an old one *with* a replica, so the controller waited for the deadline, scaled it down and removed the annotation, and the following F6 release behaved normally.

The same mechanism explains every F6 result of the matrix:

- **S3 and S5** had the same stale annotation from F5 (caught by the experiment). But F6 starts with the experiment step in these strategies, and it was stopped by the experiment's own deadline, which is evaluated by the experiment controller and does not consult this guard.
- **S2** promoted F5, and **S0** had no aborted release before F6 at all.
- The F1, F3 and F4 ReplicaSets left behind by the experiment in S3 and S5 were cleaned up again before they could matter: the same images were released and aborted by mirroring, with the canary pod running, in S4 and S6.
- **BG** uses a different Rollout, with its own ReplicaSets.

Because each fault is always the same image, its ReplicaSet is reused whenever the image is released again, which is how a stale annotation from one trial could reach a later one. In a real system every release has a new image, and the stale ReplicaSet disappears only when it falls out of the revision history, which keeps three old ReplicaSets in this platform and ten by default. Until then, *every* release of the Rollout runs without a working progress deadline.

3.6.3 Confirmation

The analysis makes a prediction that does not depend on mirroring or the experiment at all: as long as the stale annotation exists, F6 should hang under *any* canary strategy, including S1, where it was aborted after 121 s in the matrix; and once the annotation is removed, the deadline should fire again. Two additional trials, run on 3 November 2026 with the cluster still in the state the matrix had left it in, tested this prediction (Table 3.9). Their results were kept apart from the matrix data.

Trial	Stale annotation	Predicted	Measured
A: S1 + F6	present (F5 ReplicaSet, 0 replicas)	hang	no abort; still *Progressing* at step 0 after 232 s, aborted by the runner
B: S1 + F6	removed by hand before the release	deadline abort	aborted by the progress deadline after 120.6 s

Table 3.9: Confirmation of the cause of the hang. The only difference between the two trials is the `scale-down-deadline` annotation on the old, empty F5 ReplicaSet.

Both predictions held. The same strategy and the same image behaved in opposite ways depending only on an annotation on an unrelated, empty ReplicaSet, which confirms that the hang in S6 and S7 was caused by the stale annotation left by the preceding F5 trial, not by the strategies themselves.

3.6.4 Relation to known issues and consequences

The suppression itself is a known problem. Issue #4971 of Argo Rollouts [18], opened in August 2026, describes the same guard being kept false by a stale annotation on an orphaned ReplicaSet, which also makes the controller re-queue the Rollout in a tight loop. The pull request proposed to fix it [19] makes `isWaitingForReplicaSetScaleDown` ignore deadlines that have already passed; it had not been merged at the time of writing, so v1.10.1 is affected. The issue reproduces the stale annotation differently: the aborted release is left in the aborted state until its scale-down delay expires. The experiments of this work show a second, more ordinary path to the same state, which needs no waiting at all: a release aborted while its own canary ReplicaSet is empty, which is the normal situation during an experiment step, followed by a rollback through Git. A GitOps rollback is the standard reaction to an aborted release, so this path is likely to occur in practice wherever experiment steps are used.

The consequences for the platform are as follows:

- The progress deadline cannot be relied on as the only protection against a version that does not start. After an abort in an experiment step, the next releases have no deadline until the stale ReplicaSet leaves the revision history.
- The failure is silent: the Rollout stays *Progressing*, so no alert based on the *Degraded* phase fires. An alert on the duration of the *Progressing* phase would catch it.
- Until the fix is released, the stale annotation can be removed after an abort (`kubectl annotate rs <name> scale-down-deadline-`), or the platform can run a periodic job that removes the annotation from ReplicaSets with zero replicas.

3.7 Discussion

RQ1: which mechanism catches what. Each mechanism has a clear profile. Mirroring catches errors and latency fastest (in 27–49 s) and without exposing any user, but it is blind to slow faults such as the leak, because its window is short and it measures no memory. The experiment catches every metric fault, the leak included, in 47–61 s, but it always exposes 20 % of the users. The canary also catches every metric fault, but the leak only late and at 50 %. None of the metric-based mechanisms can catch a fault below its threshold (F2) or a fault that the metrics do not describe (F8). Only the smoke Job, which checks the content of the responses, caught F8, and it caught F2 only by chance.

RQ2: the cost. Protection is paid for in time and resources on every release, and the healthy release shows the price. Full protection (S7) takes 6.8 minutes and 12.7 pod-minutes per release, against 2.1 and 2.1 for the canary alone. The experiment accounts for most of the difference. In exposure, the cost is fault-specific: mirroring reduced the users exposed to loud faults to zero, the experiment doubled it, and nothing reduced the exposure to F2 and F8 in a canary release.

RQ3: absolute or relative checks. In these experiments neither approach produced a false alarm, and both caught the same faults: the relative check was not better at avoiding misses (both missed F2 and F8). The difference was in timing. Because a fresh baseline makes the same start-up allocation as the canary, the relative memory check could start

after 60 s instead of 90 s, and caught the leak at 20 % instead of 50 %. The real advantage of relative analysis, robustness against changes of the environment that affect both versions, could not be shown, because environment faults were out of scope. A relative check also has a blind spot of its own: if both versions are bad, for example because of a broken dependency, their difference is small. The absolute checks should therefore stay as a floor.

RQ4: combinations. Combining mechanisms helps only where a later mechanism catches what an earlier one misses. Mirroring before the canary (S4) moved the loud faults from 10 % to 0 % exposure, at a cost of 1.6 minutes per release. Adding the experiment to mirroring and canary (S4 to S7) moved the leak from 50 % to 20 %, for 3.1 more minutes. The canary after mirroring and the experiment (S6 to S7) changed no outcome at all: it only added 1.8 minutes. In the opposite direction, the experiment before a canary (S1 to S5) made the loud faults worse. The order and placement of the mechanisms matter as much as their selection: an experiment placed after the first small canary step, or with a lower weight, would avoid exposing more users than the canary itself.

RQ5: blue-green. Blue-green inverts the risk profile of the canary. Before the switch it exposes nobody, and a functional smoke test can catch faults that no metric shows (F8). After the switch every user is exposed, and the faults that the smoke test does not cover (latency and the leak) reached all users for 82–111 s. Its rollback was as fast as the canary's. Blue-green is therefore attractive where a meaningful functional test exists and the faults to be feared are functional ones, and weak against performance faults that appear only under real load.

Defence in depth and a decision guide. No single mechanism caught every fault, and the strategies that combine complementary mechanisms (mirroring for loud faults, a relative or long-running check for slow ones, a functional test for semantic ones) caught the most with the least exposure. The results suggest the following choices:

- a read-heavy service, where users must not see errors: mirroring before the canary (S4);
- slow faults expected: add the experiment, but after the first small canary step, not before it;
- limited resources or frequent releases: the canary alone (S1);
- faults that the metrics cannot see: a functional check, such as the smoke Job, which can also run as an analysis step of a canary;
- in every case: an alert on releases that stay *Progressing* too long, because the progress deadline cannot be relied on alone (Section 3.6).

3.8 Threats to validity

Internal validity. Each cell was run once. Most outcomes follow from the configuration (thresholds, delays, step order) and are expected to repeat, which the consistent detection times across strategies support, but the two random outcomes rest on a single sample: F2 under blue-green, caught with a probability of about 64 %, and to a lesser extent F4. The

trials ran one after the other on the same cluster, and the hang in Section 3.6 shows that state can leak from one trial into the next: the reuse of the same image, and therefore the same ReplicaSet, for every release of a fault made this possible. The other results do not depend on such state, because the metric checks query only the pods of the new version. Four measurement defects were found after the matrix; they affected only the computation of the measurements, which was repeated from the recorded timelines while Prometheus still held the data.

Measurement resolution. Prometheus scrapes every 10 s and the checks use one-minute rate windows, so detection times below about 30 s are not resolved precisely, and the runner's 2 s polling bounds the resolution of the recovery time.

External validity. The environment is small: a single node, one replica per version, and synthetic traffic of about 5 requests per second to a single endpoint. With a few requests per measurement window, the checks have little statistical power, and the thresholds were chosen for this application. The faults are deterministic and present from the start, unlike many real faults, which depend on input or appear only under load. Environment faults, where relative analysis is expected to excel, were not tested, and neither were other orders of the mechanisms, such as an experiment after the first canary step. The production environment of the platform was not used.

Future work. The experiments point to several extensions: statistical canary analysis, which compares canary and baseline with a statistical test instead of fixed thresholds, as Kayenta does [20]; testing the experiment after the first canary step and with a lower weight; environment faults; multiple replicas and nodes; and repeated runs of the cells with random outcomes.

Chapter 4

LaTeX-eszközök

4.1 A szerkesztéshez használatos eszközök

Ez a sablon TeXstudio 2.8.8 szerkesztővel készült. A TeXstudio egy platformfüggetlen, Windows, Linux és Mac OS alatt is elérhető LaTeX-szerkesztőprogram számtalan hasznos szolgáltatással (figure 4.1). A szoftver ingyenesen letölthető[1].

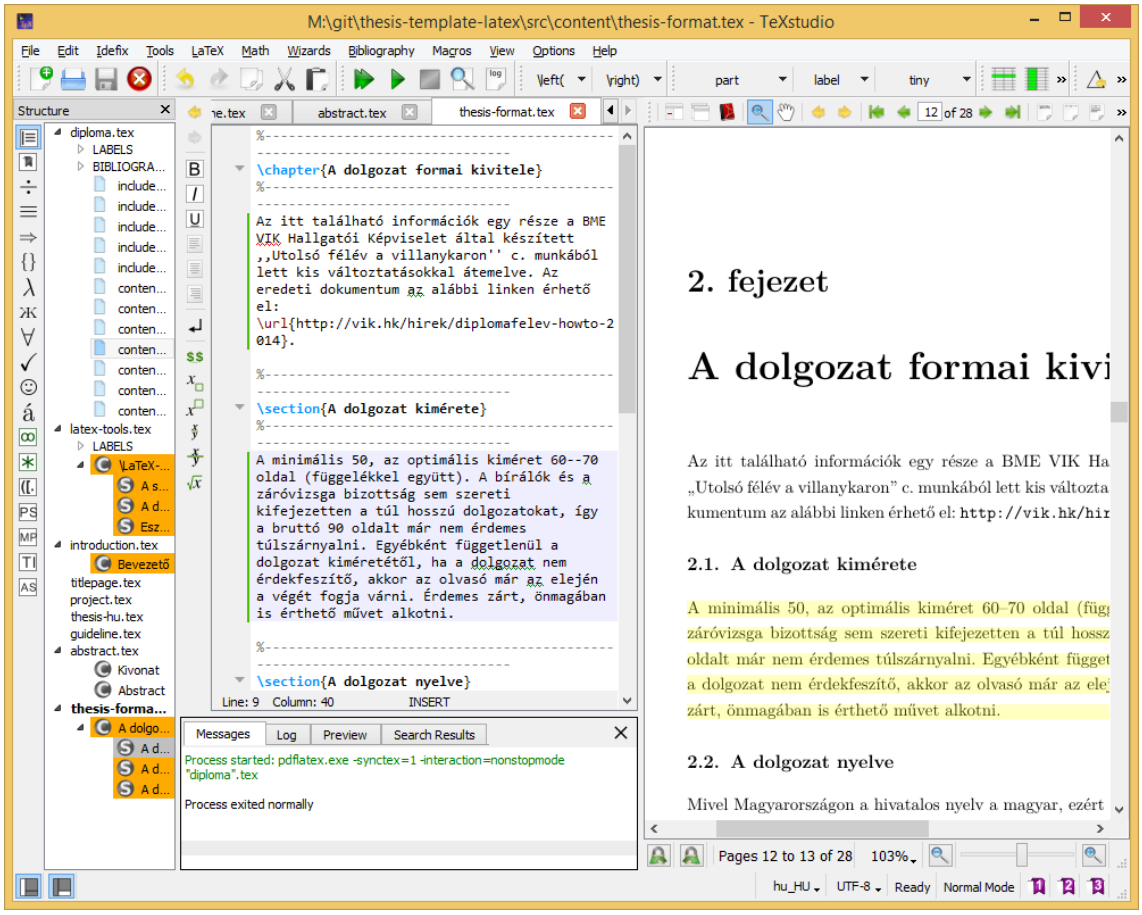


Figure 4.1: A TeXstudio LaTeX-szerkesztő.

[1] A TeXstudio hivatalos oldala: `http://texstudio.sourceforge.net/`

A TeXstudio telepítése után érdemes még letölteni a magyar nyelvű helyesírásellenőrző-szótárakat hozzá. A TeXstudio az OpenOffice-hoz használatos formátumot tudja kezelni. A TeXstudio beállításainál a `General` fülön a `Dictionaries` résznél tudjuk megadni, hogy melyik szótárat használja.

Egy másik használható Windows alapú szerkesztőprogram a LEd[2] (LaTeX Editor), a TeXstudio azonban stabilabb, gyorsabb, és jobban használható.

4.2 A dokumentum lefordítása Windows alatt

A TeXstudio és a LEd kizárólag szerkesztőprogram (bár az utóbbiban DVI-nézegető is van), így a dokumentum fordításához szükséges eszközöket nem tartalmazza. Windows alatt alapvetően két lehetőség közül érdemes választani: MiKTeX (`http://miktex.org/`) és TeX Live (`http://www.tug.org/texlive/`) programcsomag. Az utóbbi működik Mac OS X, GNU/Linux alatt és Unix-származékokon is. A MiKTeX egy alapcsomag telepítése után mindig letölti a használt funkciókhoz szükséges, de lokálisan hiányzó TEX-csomagokat, míg a TeX Live DVD ISO verzóban férhető hozzá. Ez a dokumentum TeX Live 2008 programcsomag segítségével fordult, amelynek DVD ISO verziója a megadott oldalról letölthető. A sablon lefordításához a disztribúcióban szereplő `magyar.ldf` fájlt a `http://www.math.bme.hu/latex/` változatra kell cserélni, vagy az utóbbi változatot be kell másolni a projekt-könyvtárba (ahogy ezt meg is tettük a sablonban) különben anomáliák tapasztalhatók a dokumentumban (pl. az ábra- és táblázat-aláírások formátuma nem a beállított lesz, vagy bizonyos oldalakon megjelenik alapértelmezésben egy fejléc). A TeX Live 2008-at még nem kell külön telepíteni a gépre, elegendő DVD-ről (vagy az ISO fájlból közvetlenül, pl. DaemonTools-szal) használni.

Ha a MiKTeX csomagot használjuk, akkor parancssorból a következő módon tudjuk újrafordítani a teljes dokumentumot:

```
$ texify -p thesis.tex
```

A `texify` parancs a MiKTex programcsomag `miktex/bin` alkönyvtárában található. A parancs gondoskodik arról, hogy a szükséges lépéseket (fordítás, hivatkozások generálása stb.) a megfelelő sorrendben elvégezze. A `-p` kapcsoló hatására PDF-et generál. A fordítást és az ideiglenes fájlok törlését elvégezhetjük a sablonhoz mellékelt `manual_build.bat` szkript segítségével is.

A TEX-eszközöket tartalmazó programcsomag binárisainak elérési útját gyakran be kell állítani a szerkesztőprogramban, például TeXstudio esetén legegyszerűbben az `Options / Configure TeXstudio... / Commands` menüponttal előhívott dialógusablakban tehetjük ezt meg.

A PDF-LATEX használata esetén a generált dokumentum közvetlenül PDF-formátumban áll rendelkezésre. Amennyiben a PDF-fájl egy PDF-nézőben (pl. Adobe Acrobat Reader vagy Foxit PDF Reader) meg van nyitva, akkor a fájlleírót a PDF-néző program tipikusan lefoglalja. Ilyen esetben a dokumentum újrafordítása hibaüzenettel kilép. Ha bezárjuk és újra megnyitjuk a PDF dokumentumot, akkor pedig a PDF-nézők többsége az első oldalon nyitja meg a dokumentumot, nem a legutóbb olvasott oldalon. Ezzel szemben például az egyszerű és ingyenes Sumatra PDF nevű program képes arra, hogy a megnyitott dokumentum megváltozását detektálja, és frissítse a nézetet az aktuális oldal megtartásával.

[2] A LEd hivatalos oldala: `http://www.latexeditor.org/`

4.3 Eszközök Linuxhoz

Linux operációs rendszer alatt is rengeteg szerkesztőprogram van, pl. a KDE alapú Kile jól használható. Ez ingyenesen letölthető, vagy éppenséggel az adott Linux-disztribúció eleve tartalmazza, ahogyan a dokumentum fordításához szükséges csomagokat is. Az Ubuntu Linux disztribúciók alatt például legtöbbször a `texlive-*` csomagok telepítésével használhatók a LaTeX-eszközök. A jelen sablon fordításához szükséges csomagok (kb. 0,5 GB) az alábbi paranccsal telepíthetők:

```
$ sudo apt-get install texlive-latex-extra texlive-fonts-extra texlive-fonts-recommended
    texlive-xetex texlive-science
```

Amennyiben egy újabb csomag hozzáadása után hiányzó fájlra utaló hibát kapunk a fordítótól, telepítenünk kell az azt tartalmazó TeX Live csomagot. Ha pl. a `bibentry` csomagot szeretnénk használni, futtassuk az alábbi parancsot:

```
$ apt-cache search bibentry
texlive-luatex - TeX Live: LuaTeX packages
```

Majd telepítsük fel a megfelelő TeX Live csomagot, jelen esetben a ‘texlive-lualatex‘-et. (Egy LaTeX csomag több TeX Live csomagban is szerepelhet.)

Ha gyakran szerkesztünk más LaTeXdokumentumokat is, kényelmes és biztos megoldás a teljes TeX Live disztribúció telepítése, ez azonban kb. 4 GB helyet igényel.

```
sudo apt-get install texlive-full
```

Chapter 5

A dolgozat formai kivitele

Az itt található információk egy része a BME VIK Hallgatói Képviselet által készített „Utolsó félév a villanykaron” c. munkából lett kis változtatásokkal átemelve. Az eredeti dokumentum az alábbi linken érhető el: `http://vik.hk/hirek/diplomafelev-howto-2015`.

5.1 A dolgozat kimérete

Szakdolgozat esetében minimum 30, 45 körüli ajánlott oldalszám lehet az iránymutató. De mindenképp érdemes rákérdezni a konzulensnél is az elvárásokra, mert tanszékenként változóak lehetnek az elvárások.

Mesterképzésen a Diplomatervezés 1 esetében a beszámoló még inkább az Önálló laboratóriumi beszámolókhoz hasonlít, tanszékenként eltérő formai követelményekkel, – egy legalább 30 oldal körüli dolgozat az elvárt – és az elmúlt fél éves munkáról szól. De egyben célszerű, ha ez a végleges diplomaterv alapja is. (A végleges 60-90 oldal körülbelül a hasznos részre nézve)

5.2 A dolgozat nyelve

Mivel Magyarországon a hivatalos nyelv a magyar, ezért alapértelmezésben magyarul kell megírni a dolgozatot. Aki külföldi posztgraduális képzésben akar részt venni, nemzetközi szintű tudományos kutatást szeretne végezni, vagy multinacionális cégnél akar elhelyezkedni, annak célszerű angolul megírnia diplomadolgozatát. Mielőtt a hallgató az angol nyelvű verzió mellett dönt, erősen ajánlott mérlegelni, hogy ez mennyi többletmunkát fog a hallgatónak jelenteni fogalmazás és nyelvhelyesség terén, valamint – nem utolsó sorban – hogy ez mennyi többletmunkát fog jelenteni a konzulens illetve bíráló számára. Egy nehezen olvasható, netalán érthetetlen szöveg teher minden játékos számára.

5.3 A dokumentum nyomdatechnikai kivitele

A dolgozatot A4-es fehér lapra nyomtatva, 2,5 centiméteres margóval (+1 cm kötésbeni), 11–12 pontos betűmérettel, talpas betűtípussal és másfeles sorközzel célszerű elkészíteni.

Annak érdekében, hogy a dolgozat külsőleg is igényes munka benyomását keltse, érdemes figyelni az alapvető tipográfiai szabályok betartására [21].

Chapter 6

A LaTeX-sablon használata

Ebben a fejezetben röviden, implicit módon bemutatjuk a sablon használatának módját, ami azt jelenti, hogy sablon használata ennek a dokumentumnak a forráskódját tanulmányozva válik teljesen világossá. Amennyiben a szoftver-keretrendszer telepítve van, a sablon alkalmazása és a dolgozat szerkesztése LaTeX-ben a sablon segítségével tapasztalataink szerint jóval hatékonyabb, mint egy WYSWYG (*What You See is What You Get*) típusú szövegszerkesztő esetén (pl. Microsoft Word, OpenOffice).

6.1 Címkék és hivatkozások

A LaTeX dokumentumban címkéket (`\label`) rendelhetünk ábrákhoz, táblázatokhoz, fejezetekhez, listákhoz, képletekhez stb. Ezekre a dokumentum bármely részében hivatkozhatunk, a hivatkozások automatikusan feloldásra kerülnek.

A sablonban makrókat definiáltunk a hivatkozások megkönnyítéséhez. Ennek megfelelően minden ábra (*figure*) címkéje `fig:` kulcsszóval kezdődik, míg minden táblázat (*table*), képlet (*equation*), fejezet (*section*) és lista (*listing*) rendre a `tab:`, `eq:`, `sec:` és `lst:` kulcsszóval kezdődik, és a kulcsszavak után tetszőlegesen választott címke használható. Ha ezt a konvenciót betartjuk, akkor az előbbi objektumok számára rendre a `\figref`, `\tabref`, `\eqref`, `\sectref` és `\listref` makrókkal hivatkozhatunk. A makrók paramétere a címke, amelyre hivatkozunk (a kulcsszó nélkül). Az összes említett hivatkozástípus, beleértve az `\url` kulcsszóval bevezetett web-hivatkozásokat is a `hyperref`[1] csomagnak köszönhetően aktívak a legtöbb PDF-nézegetőben, rájuk kattintva a dokumentum megfelelő oldalára ugrik a PDF-néző vagy a megfelelő linket megnyitja az alapértelmezett böngészővel. A `hyperref` csomag a kimeneti PDF-dokumentumba könyvjelzőket is készít a tartalomjegyzékből. Ez egy szintén aktív tartalomjegyzék, amelynek elemeire kattintva a nézegető behozza a kiválasztott fejezetet.

6.2 Ábrák és táblázatok

Használjunk vektorgrafikus ábrákat, ha van rá módunk. PDFLaTeX használata esetén PDF formátumú ábrákat lehet beilleszteni könnyen, az EPS (PostScript) vektorgrafikus képformátum beillesztését a PDFLaTeX közvetlenül nem támogatja (de lehet konvertálni,

[1] Segítségével a dokumentumban megjelenő hivatkozások nem csak dinamikussá válnak, de színezhetők is, bővebbet erről a csomag dokumentációjában találunk. Ez egyúttal egy példa lábjegyzet írására.

lásd később). Ha vektorgrafikus formában nem áll rendelkezésünkre az ábra, akkor a veszteségmentes PNG, valamint a veszteséges JPEG formátumban érdemes elmenteni. Figyeljünk arra, hogy ilyenkor a képek felbontása elég nagy legyen ahhoz, hogy nyomtatásban is megfelelő minőséget nyújtson (legalább 300 dpi javasolt). A dokumentumban felhasznált képfájlokat a dokumentum forrása mellett érdemes tartani, archiválni, mivel ezek hiányában a dokumentum nem fordul újra. Ha lehet, a vektorgrafikus képeket vektorgrafikus formátumban is érdemes elmenteni az újrafelhasználhatóság (az átszerkeszthetőség) érdekében.

Kapcsolási rajzok legtöbbször kimásolhatók egy vektorgrafikus programba (pl. CorelDraw) és onnan nagyobb felbontással raszterizálva kimenthatők PNG formátumban. Ugyanakkor kiváló ábrák készíthetők Microsoft Visio vagy hasonló program használatával is: Visio-ból az ábrák közvetlenül PDF-be is menthetők.

Lehetőségeink Matlab ábrák esetén:

- Képernyőlopás (*screenshot*) is elfogadható minőségű lehet a dokumentumban, de általában jobb felbontást is el lehet érni más módszerrel.
- A Matlab ábrát a `File/Save As` opcióval lementhetjük PNG formátumban (ugyanaz itt is érvényes, mint korábban, ezért nem javasoljuk).
- A Matlab ábrát az `Edit/Copy figure` opcióval kimásolhatjuk egy vektorgrafikus programba is és onnan nagyobb felbontással raszterizálva kimenthatjük PNG formátumban (nem javasolt).
- Javasolt megoldás: az ábrát a `File/Save As` opcióval EPS *vektorgrafikus* formátumban elmentjük, PDF-be konvertálva beillesztjük a dolgozatba.

Az EPS kép az `epstopdf` programmal[2] konvertálható PDF formátumba. Célszerű egy batch-fájlt készíteni az összes EPS ábra lefordítására az alábbi módon (ez Windows alatt működik).

```
@echo off
for %%j in (*.eps) do (
  echo converting file "%%j"
  epstopdf "%%j"
)
echo done .
```

Egy ilyen parancsfájlt (`convert.cmd`) elhelyeztük a sablon `figures\eps` könyvtárába, így a felhasználónak csak annyi a dolga, hogy a `figures\eps` könyvtárba kimenti az EPS formátumú vektorgrafikus képet, majd lefuttatja a `convert.cmd` parancsfájlt, ami PDF-be konvertálja az EPS fájlt.

Ezek után a PDF-ábrát ugyanúgy lehet a dokumentumba beilleszteni, mint a PNG-t vagy a JPEG-et. A megoldás előnye, hogy a lefordított dokumentumban is vektorgrafikusan tárolódik az ábra, így a mérete jóval kisebb, mintha raszterizáltuk volna beillesztés előtt. Ez a módszer minden – az EPS formátumot ismerő – vektorgrafikus program (pl. CorelDraw) esetén is használható.

A képek beillesztésére a chapter 4ben mutattunk be példát (figure 4.1). Az előző mondatban egyúttal az automatikusan feloldódó ábrahivatkozásra is láthatunk példát. Több képfájlt is beilleszthetünk egyetlen ábrába. Az egyes képek közötti horizontális és vertikális margót metrikusan szabályozhatjuk (figure 6.1). Az ábrák elhelyezését számtalan

[2] a korábban említett LaTeX-disztribúciókban megtalálható

tipográfiai szabály egyidejű teljesítésével a fordító maga végzi, a dokumentum írója csak preferenciáit jelezheti a fordító felé (olykor ez bosszúságot is okozhat, ilyenkor pl. a kép méretével lehet játszani).

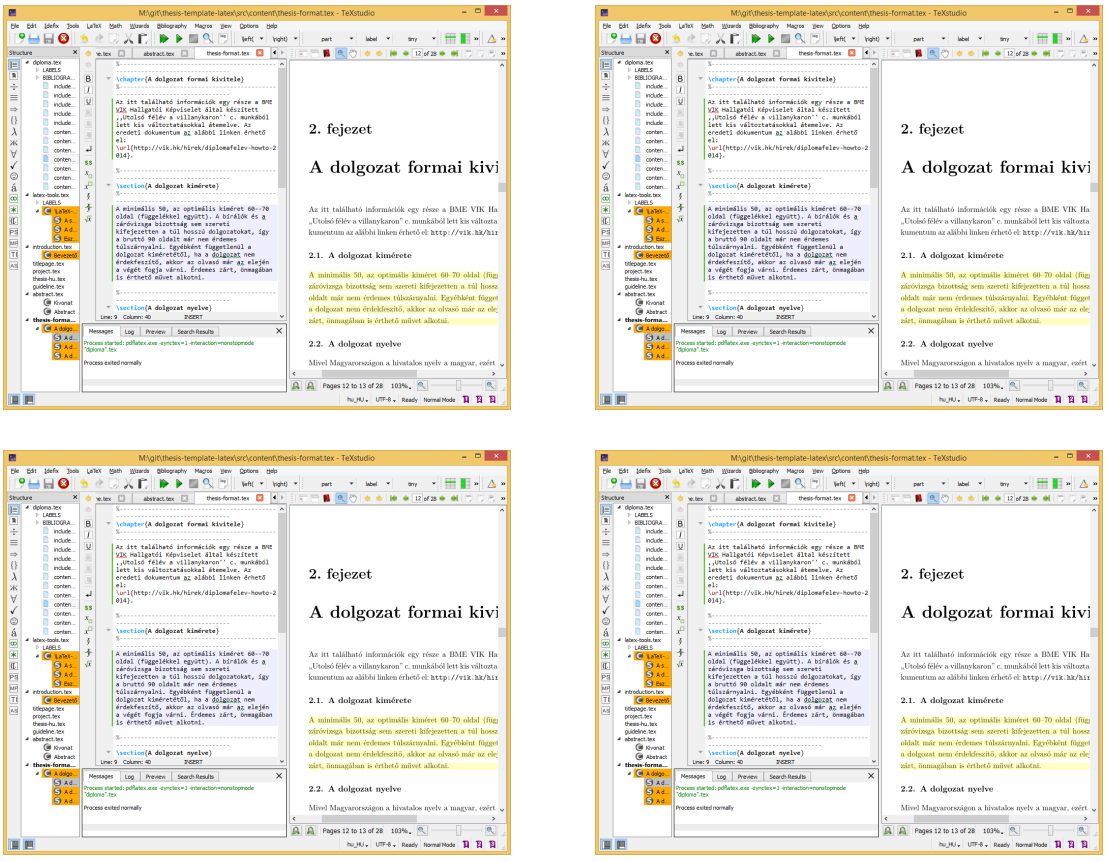


Figure 6.1: Több képfájl beillesztése esetén térközöket is érdemes használni.

A táblázatok használatára a 6.1 táblázat mutat példát. A táblázatok formázásához hasznos tanácsokat találunk a `booktabs` csomag dokumentációjában.

Órajel	Frekvencia	Cél pin
CLKA	100 MHz	FPGA CLK0
CLKB	48 MHz	FPGA CLK1
CLKC	20 MHz	Processzor
CLKD	25 MHz	Ethernet chip
CLKE	72 MHz	FPGA CLK2
XBUF	20 MHz	FPGA CLK3

Table 6.1: Az órajel-generátor chip órajel-kimenetei.

6.3 Felsorolások és listák

Számozatlan felsorolásra mutat példát a jelenlegi bekezdés:

- *első bajusz:* ide lehetne írni az első elem kifejését,
- *második bajusz:* ide lehetne írni a második elem kifejését,

- *ez meg egy szakáll:* ide lehetne írni a harmadik elem kifejését.

Számozott felsorolást is készíthetünk az alábbi módon:

1. *első bajusz:* ide lehetne írni az első elem kifejését, és ez a kifejtés így néz ki, ha több sorosra sikeredik,
2. *második bajusz:* ide lehetne írni a második elem kifejését,
3. *ez meg egy szakáll:* ide lehetne írni a harmadik elem kifejését.

A felsorolásokban sorok végén vessző, az utolsó sor végén pedig pont a szokásos írásjel. Ez alól kivételt képezhet, ha az egyes elemek több teljes mondatot tartalmaznak.

Listákban a dolgozat szövegétől elkülönítendő kódrészleteket, programsorokat, pszeudokódokat jeleníthetünk meg (6.1. kódrészlet).

```
\begin{enumerate}
  \item \emph{első bajusz:} ide lehetne írni az első elem kifejését,
  és ez a kifejtés így néz ki, ha több sorosra sikeredik,
  \item \emph{második bajusz:} ide lehetne írni a második elem kifejését,
  \item \emph{ez meg egy szakáll:} ide lehetne írni a harmadik elem kifejését.
\end{enumerate}
```

Listing 6.1: A fenti számozott felsorolás LaTeX-forráskódja

A lista keretét, háttérszínét, egész stílusát megválaszthatjuk. Ráadásul különféle programnyelveket és a nyelveken belül kulcsszavakat is definiálhatunk, ha szükséges. Erről bővebbet a `listings` csomag hivatalos leírásában találhatunk.

6.4 Képletek

Ha egy formula nem túlságosan hosszú, és nem akarjuk hivatkozni a szövegből, mint például a $e^{i\pi} + 1 = 0$ képlet, *szövegközi képletként* szokás leírni. Csak, hogy másik példát is lássunk, az $U_i = -d\Phi/dt$ Faraday-törvény a $\operatorname{rot} E = -\frac{dB}{dt}$ differenciális alakban adott Maxwell-egyenlet felületre vett integráljából vezethető le. Látható, hogy a LaTeX-fordító a sorközöket betartja, így a szöveg szedése esztétikus marad szövegközi képletek használata esetén is.

Képletek esetén az általános konvenció, hogy a kisbetűk skalárt, a kis félkövér betűk ($\mathbf{v}$) oszlopvektort – és ennek megfelelően $\mathbf{v}^T$ sorvektort – a kapitális félkövér betűk ($\mathbf{V}$) mátrixot jelölnek. Ha ettől el szeretnénk térni, akkor az alkalmazni kívánt jelölésmódot célszerű külön alfejezetben definiálni. Ennek megfelelően, amennyiben $\mathbf{y}$ jelöli a mérések vektorát, ϑ a paraméterek vektorát és $\hat{\mathbf{y}} = \mathbf{X}\vartheta$ a paraméterekben lineáris modellt, akkor a *Least-Squares* értelemben optimális paraméterbecslő $\hat{\vartheta}_{LS} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ lesz.

Emellett kiemelt, sorszámozott képleteket is megadhatunk, ennél az `equation` és a `eqnarray` környezetek helyett a korszerűbb `align` környezet alkalmazását javasoljuk (több okból, különféle problémák elkerülése végett, amelyekre most nem térünk ki). Tehát

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}, \tag{6.1}$$

$$\mathbf{y} = \mathbf{C}\mathbf{x}, \tag{6.2}$$

ahol $\mathbf{x}$ az állapotvektor, $\mathbf{y}$ a mérések vektora és $\mathbf{A}$, $\mathbf{B}$ és $\mathbf{C}$ a rendszert leíró paramétermátrixok. Figyeljük meg, hogy a két egyenletben az egyenlőségjelek egymáshoz igazítva

jelennek meg, mivel a mindkettőt az & karakter előzi meg a kódban. Lehetőség van számozatlan kiemelt képlet használatára is, például

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u},$$
$$\mathbf{y} = \mathbf{C}\mathbf{x}.$$

Mátrixok felírására az $\mathbf{A}\mathbf{x} = \mathbf{b}$ inhomogén lineáris egyenlet részletes kifejtésével mutatunk példát:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}. \tag{6.3}$$

A `\frac` utasítás hatékonyságát egy általános másodfokú tag átviteli függvényén keresztül mutatjuk be, azaz

$$W(s) = \frac{A}{1 + 2T\xi s + s^2T^2}. \tag{6.4}$$

A matematikai mód minden szimbólumának és képességének a bemutatására természetesen itt nincs lehetőség, de gyors referenciaként hatékonyan használhatók a következő linkek:
`http://www.artofproblemsolving.com/LaTeX/AoPS_L_GuideSym.php`,
`http://www.ctan.org/tex-archive/info/symbols/comprehensive/symbols-a4.pdf`,
`ftp://ftp.ams.org/pub/tex/doc/amsmath/short-math-guide.pdf`.
Ez pedig itt egy magyarázat, hogy miért érdemes `align` környezetet használni:
`http://texblog.net/latex-archive/maths/eqnarray-align-environment/`.

6.5 Irodalmi hivatkozások

Egy LaTeX dokumentumban az irodalmi hivatkozások definíciójának két módja van. Az egyik a `\thebibliograhy` környezet használata a dokumentum végén, az `\end{document}` lezárás előtt.

```
\begin{thebibliography}{9}

\bibitem{Lamport94} Leslie Lamport, \emph{\LaTeX: A Document Preparation System}.
Addison Wesley, Massachusetts, 2nd Edition, 1994.

\end{thebibliography}
```

Ezek után a dokumentumban a `\cite{Lamport94}` utasítással hivatkozhatunk a forrásra. A fenti megadás viszonylag kötetlen, a szerző maga formázza az irodalomjegyzéket (ami gyakran inkonzisztens eredményhez vezet).

Egy sokkal professzionálisabb módszer a BiBTeX használata, ezért ez a sablon is ezt támogatja. Ebben az esetben egy külön szöveges adatbázisban definiáljuk a forrásmunkákat, és egy külön stílusfájl határozza meg az irodalomjegyzék kinézetét. Ez, összhangban azzal, hogy külön formátumkonvenció határozza meg a folyóirat-, a könyv-, a konferenciacikk- stb. hivatkozások kinézetét az irodalomjegyzékben (a sablon használata esetén ezzel nem is kell foglalkoznia a hallgatónak, de az eredményt célszerű ellenőrizni). felhasznált hivatkozások adatbázisa egy `.bib` kiterjesztésű szöveges fájl, amelynek szerkezetét a A 6.2 kódrészlet demonstrálja. A forrásmunkák bevitelekor a sor végi vesszők külön figyelmet igényelnek, mert hiányuk a BiBTeX-fordító hibaüzenetét eredményezi. A forrásmunkákat

típus szerinti kulcsszó vezeti be (`@book` könyv, `@inproceedings` konferenciakiadványban megjelent cikk, `@article` folyóiratban megjelent cikk, `@techreport` valamelyik egyetem gondozásában megjelent műszaki tanulmány, `@manual` műszaki dokumentáció esetén stb.). Nemcsak a megjelenés stílusa, de a kötelezően megadandó mezők is típusról-típusra változnak. Egy jól használható referencia a `http://en.wikipedia.org/wiki/BibTeX` oldalon található.

```
@book{Wettl04,
  author    = {Ferenc Wettl and Gyula Mayer and Péter Szabó},
  publisher = {Panem Könyvkiadó},
  title     = {\LaTeX~kézikönyv},
  year      = {2004},
}

@article{Candy86,
  author       = {James C. Candy},
  journaltitle = {{IEEE} Trans.\ on Communications},
  month        = {01},
  note         = {\doi{10.1109/TCOM.1986.1096432}},
  number       = {1},
  pages        = {72--76},
  title        = {Decimation for Sigma Delta Modulation},
  volume       = {34},
  year         = {1986},
}

@inproceedings{Lee87,
  author    = {Wai L. Lee and Charles G. Sodini},
  booktitle = {Proc.\ of the IEEE International Symposium on Circuits and Systems},
  location  = {Philadelphia, PA, USA},
  month     = {05~4--7},
  pages     = {459--462},
  title     = {A Topology for Higher Order Interpolative Coders},
  vol       = {2},
  year      = {1987},
}

@thesis{KissPhD,
  author      = {Peter Kiss},
  institution = {Technical University of Timi\c{s}oara, Romania},
  month       = {04},
  title       = {Adaptive Digital Compensation of Analog Circuit Imperfections for Cascaded Delta-
    Sigma Analog-to-Digital Converters},
  type        = {phdthesis},
  year        = {2000},
}

@manual{Schreier00,
  author       = {Richard Schreier},
  month        = {01},
  note         = {\url{http://www.mathworks.com/matlabcentral/fileexchange/}},
  organization = {Oregon State University},
  title        = {The Delta-Sigma Toolbox v5.2},
  year         = {2000},
}

@misc{DipPortal,
  author       = {{Budapesti űMszaki és Gazdaságtudományi Egyetem Villamosmérnöki és Informatikai
    Kar}},
  howpublished = {\url{http://diplomaterv.vik.bme.hu/}},
  title        = {Diplomaterv portál (2011. február 26.)},
}

@incollection{Mkrtychev:1997,
  author    = {Mkrtychev, Alexey},
  booktitle = {Logical Foundations of Computer Science},
  doi       = {10.1007/3-540-63045-7_27},
  editor    = {Adian, Sergei and Nerode, Anil},
  isbn      = {978-3-540-63045-6},
  pages     = {266-275},
  publisher = {Springer Berlin Heidelberg},
```

```
  series    = {Lecture Notes in Computer Science},
  title     = {Models for the logic of proofs},
  url       = {http://dx.doi.org/10.1007/3-540-63045-7_27},
  volume    = {1234},
  year      = {1997},
}
```

Listing 6.2: Példa szöveges irodalomjegyzék-adatbázisra BibTEX használata esetén.

A stílusfájl egy `.sty` kiterjesztésű fájl, de ezzel lényegében nem kell foglalkozni, mert vannak beépített stílusok, amelyek jól használhatók. Ez a sablon a BiBTEX-et használja, a hozzá tartozó adatbázisfájl a `mybib.bib` fájl. Megfigyelhető, hogy az irodalomjegyzéket a dokumentum végére (a `\end{document}` utasítás elé) beillesztett `\bibliography{mybib}` utasítással hozhatjuk létre, a stílusát pedig ugyanitt a `\bibliographystyle{plain}` utasítással adhatjuk meg. Ebben az esetben a `plain` előre definiált stílust használjuk (a sablonban is ezt állítottuk be). A `plain` stíluson kívül természetesen számtalan más előre definiált stílus is létezik. Mivel a `.bib` adatbázisban ezeket megadtuk, a BiBTEX-fordító is meg tudja különböztetni a szerzőt a címtől és a kiadótól, és ez alapján automatikusan generálódik az irodalomjegyzék a stílusfájl által meghatározott stílusban.

Az egyes forrásmunkákra a szövegből továbbra is a `\cite` paranccsal tudunk hivatkozni, így a 6.2. kódrészlet esetén a hivatkozások rendre `\cite{Wettl04}`, `\cite{Candy86}`, `\cite{Lee87}`, `\cite{KissPhD}`, `\cite{Schreirer00}`, `\cite{Mkrtychev:1997}` és `\cite{DipPortal}`. Az egyes forrásmunkák sorszáma az irodalomjegyzék bővítésekor változhat. Amennyiben az aktuális számhoz illeszkedő névelőt szeretnénk használni, használjuk az `\acite{}` parancsot.

Az irodalomjegyzékben alapértelmezésben csak azok a forrásmunkák jelennek meg, amelyekre található hivatkozás a szövegben, és ez így alapvetően helyes is, hiszen olyan forrásmunkákat nem illik az irodalomjegyzékbe írni, amelyekre nincs hivatkozás.

Mivel a fordítási folyamat során több lépésben oldódnak fel a szimbólumok, ezért gyakran többször is le kell fordítani a dokumentumot. Ilyenkor ez első 1-2 fordítás esetleg szimbólum-feloldásra vonatkozó figyelmeztető üzenettel zárul. Ha hibaüzenettel zárul bármelyik fordítás, akkor nincs értelme megismételni, hanem a hibát kell megkeresni. A `.bib` fájl megváltoztatáskor sokszor nincs hatása a változtatásnak azonnal, mivel nem mindig fut újra a BibTeX fordító. Ezért célszerű a változtatás után azt manuálisan is lefuttatni (TeXstudio esetén `Tools/Bibliography`).

Hogy a szövegbe ágyazott hivatkozások kinézetét demonstráljuk, itt most sorban meghivatkozzuk a [22], [23], [24], [25], [26] és a [27][3] forrásmunkát, valamint a [28] weboldalt.

Megjegyzendő, hogy az ékezetes magyar betűket is tartalmazó `.bib` fájl az `inputenc` csomaggal betöltött `latin2` betűkészlet miatt fordítható. Ugyanez a `.bib` fájl hibaüzenettel fordul egy olyan dokumentumban, ami nem tartalmazza a `\usepackage[latin2]{inputenc}` sort. Speciális igény esetén az irodalmi adatbázis általánosabb érvényűvé tehető, ha az ékezetes betűket speciális latex karakterekkel helyettesítjük a `.bib` fájlban, pl. á helyett `\'{a}`-t vagy ő helyett `\H{o}`-t írunk.

Irodalomhivatkozásokat célszerű először olyan szolgáltatásokban keresni, ahol jó minőségű bejegyzések találhatók (pl. ACM Digital Library,[4] DBLP,[5] IEEE Xplore,[6] SpringerLink[7])

[3] Informatikai témában gyakran hivatkozunk cikkeket a Springer LNCS valamely kötetéből, ez a hivatkozás erre mutat egy helyes példát.

[4] `https://dl.acm.org/`

[5] `http://dblp.uni-trier.de/`

[6] `http://ieeexplore.ieee.org/`

[7] `https://link.springer.com/`

és csak ezek után használni kevésbé válogatott forrásokat (pl. Google Scholar[8]). A jó minőségű bejegyzéseket is érdemes megfelelően tisztítani.[9] A sablon angol nyelvű változatában használt `plainnat` beállítás egyik sajátossága, hogy a cikkhez generált hivatkozás a cikk DOI-ját és URL-jét is tartalmazza, ami gyakran duplikátumhoz vezet – érdemes tehát a DOI-kat tartalmazó URL mezőket törölni.

6.6 A dolgozat szerkezete és a forrásfájlok

A diplomatervsablonban a TeX fájlok két alkönyvtárban helyezkednek el. Az `include` könyvtárban azok szerepelnek, amiket tipikusan nem kell szerkesztenünk, ezek a sablon részei (pl. címoldal). A `content` alkönyvtárban pedig a saját munkánkat helyezhetjük el. Itt érdemes az egyes fejezeteket külön TEX állományokba rakni.

A diplomatervsablon (a kari irányelvek szerint) az alábbi fő fejezetekből áll:

1. 1 oldalas *tájékoztató* a szakdolgozat/diplomaterv szerkezetéről (`include/guideline.tex`), ami a végső dolgozatból törlendő,
2. *feladatkiírás* (`include/project.tex`), a dolgozat nyomtatott verzójában ennek a helyére kerül a tanszék által kiadott, a tanszékvezető által aláírt feladatkiírás, a dolgozat elektronikus verziójába pedig a feladatkiírás egyáltalán ne kerüljön bele, azt külön tölti fel a tanszék a diplomaterv-honlapra,
3. *címoldal* (`include/titlepage.tex`),
4. *tartalomjegyzék* (`thesis.tex`),
5. a diplomatervező *nyilatkozat*a az önálló munkáról (`include/declaration.tex`),
6. 1-2 oldalas tartalmi *összefoglaló* magyarul és angolul, illetve elkészíthető még további nyelveken is (`content/abstract.tex`),
7. *bevezetés*: a feladat értelmezése, a tervezés célja, a feladat indokoltsága, a diplomaterv felépítésének rövid összefoglalása (`content/introduction.tex`),
8. sorszámmal ellátott *fejezetek*: a feladatkiírás pontosítása és részletes elemzése, előzmények (irodalomkutatás, hasonló alkotások), az ezekből levonható következtetések, a tervezés részletes leírása, a döntési lehetőségek értékelése és a választott megoldások indoklása, a megtervezett műszaki alkotás értékelése, kritikai elemzése, továbbfejlesztési lehetőségek,
9. esetleges *köszönetnyilvánítás*ok (`content/acknowledgement.tex`),
10. részletes és pontos *irodalomjegyzék* (ez a sablon esetében automatikusan generálódik a `thesis.tex` fájlban elhelyezett `\bibliography` utasítás hatására, a section 6.5ban leírtak szerint),
11. *függelékek* (`content/appendices.tex`).

A sablonban a fejezetek a `thesis.tex` fájlba vannak beillesztve `\include` utasítások segítségével. Lehetőség van arra, hogy csak az éppen szerkesztés alatt álló `.tex` fájlt fordítsuk le, ezzel lerövidítve a fordítási folyamatot. Ezt a lehetőséget az alábbi kódrészlet biztosítja a `thesis.tex` fájlban.

[8] `http://scholar.google.com/`

[9] `https://github.com/FTSRG/cheat-sheets/wiki/BibTeX-Fixing-entries-from-common-sources`

```
\includeonly{
  guideline,%
  project,%
  titlepage,%
  declaration,%
  abstract,%
  introduction,%
  chapter1,%
  chapter2,%
  chapter3,%
  acknowledgement,%
  appendices,%
}
```

Ha az alábbi kódrészletben az egyes sorokat a `%` szimbólummal kikommentezzük, akkor a megfelelő `.tex` fájl nem fordul le. Az oldalszámok és a tartalomjegyék természetesen csak akkor billennek helyre, ha a teljes dokumentumot lefordítjuk.

6.7 Alapadatok megadása

A diplomaterv alapadatait (cím, szerző, konzulens, konzulens titulusa) a `thesis.tex` fájlban lehet megadni.

6.8 Új fejezet írása

A főfejezetek külön `content` könyvtárban foglalnak helyet. A sablonhoz 3 fejezet készült. További főfejezeteket úgy hozhatunk létre, ha új TEX fájlt készítünk a fejezet számára, és a `thesis.tex` fájlban, a `\include` és `\includeonly` utasítások argumentumába felvesszük az új `.tex` fájl nevét.

6.9 Definíciók, tételek, példák

Definition 1 (Fluxuskondenzátor térerőssége). Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. ▪

Example 1. *Példa egy példára. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.*

Theorem 1 (Kovács tétele). Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. ▪

Acknowledgements

Ez nem kötelező, akár törölhető is. Ha a szerző szükségét érzi, itt lehet köszönetet nyilvánítani azoknak, akik hozzájárultak munkájukkal ahhoz, hogy a hallgató a szakdolgozatban vagy diplomamunkában leírt feladatokat sikeresen elvégezze. A konzulensnek való köszönetnyilvánítás sem kötelező, a konzulensnek hivatalosan is dolga, hogy a hallgatót konzultálja.

Bibliography

[1] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

[2] The Kubernetes Authors. Deployments. `https://kubernetes.io/docs/concepts/workloads/controllers/deployment/`, 2026. Kubernetes documentation. Accessed: 2026-10-26.

[3] Betsy Beyer, Niall Richard Murphy, David K. Rensin, Kent Kawahara, and Stephen Thorne. *The Site Reliability Workbook: Practical Ways to Implement SRE*. O'Reilly Media, 2018.

[4] The Argo Project Authors. Argo rollouts – kubernetes progressive delivery controller. `https://argo-rollouts.readthedocs.io/en/stable/`, 2026. Argo Rollouts documentation. Accessed: 2026-10-26.

[5] Istio Authors. Virtual service. `https://istio.io/latest/docs/reference/config/networking/virtual-service/`, 2026. Istio documentation. Accessed: 2026-10-26.

[6] Prometheus Authors. Querying prometheus. `https://prometheus.io/docs/prometheus/latest/querying/basics/`, 2026. Prometheus documentation. Accessed: 2026-10-26.

[7] The Argo Project Authors. Argo CD – declarative GitOps CD for Kubernetes. `https://argo-cd.readthedocs.io/en/stable/`, 2026. Argo CD documentation: declarative GitOps continuous delivery for Kubernetes. Accessed: 2026-11-03.

[8] The Argo Project Authors. Experiment CRD. `https://argo-rollouts.readthedocs.io/en/stable/features/experiment/`, 2026. Argo Rollouts documentation of the Experiment resource and the experiment step. Accessed: 2026-11-03.

[9] The Argo Project Authors. BlueGreen deployment strategy. `https://argo-rollouts.readthedocs.io/en/stable/features/bluegreen/`, 2026. Argo Rollouts documentation of the blue-green strategy. Accessed: 2026-11-03.

[10] The Argo Project Authors. Job metrics. `https://argo-rollouts.readthedocs.io/en/stable/analysis/job/`, 2026. Argo Rollouts documentation of the Kubernetes Job analysis provider. Accessed: 2026-11-03.

[11] The Kubernetes Authors. Sidecar containers. `https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/`, 2026. Kubernetes documentation of native sidecar containers. Accessed: 2026-11-03.

[12] The Argo Project Authors. Rollout specification. https://argo-rollouts.readthedocs.io/en/stable/features/specification/, 2026. Argo Rollouts documentation of the Rollout specification, including progressDeadlineSeconds and progressDeadlineAbort. Accessed: 2026-11-03.

[13] Douglas C. Montgomery. *Design and Analysis of Experiments*. Wiley, 9th edition, 2017.

[14] The Kubernetes Authors. Kustomize components. https://kubectl.docs.kubernetes.io/guides/config_management/components/, 2026. Kustomize documentation of reusable configuration components. Accessed: 2026-11-03.

[15] Istio Authors. Istio standard metrics. https://istio.io/latest/docs/reference/config/metrics/, 2026. Istio documentation of the standard request metrics and their labels. Accessed: 2026-11-03.

[16] Prometheus Authors. Histograms and summaries. https://prometheus.io/docs/practices/histograms/, 2026. Prometheus documentation on histograms, buckets and quantile estimation. Accessed: 2026-11-03.

[17] The Argo Project Authors. Argo rollouts v1.10.1 source code. https://github.com/argoproj/argo-rollouts/tree/v1.10.1, 2026. Source code of the Argo Rollouts controller, release v1.10.1 (files rollout/sync.go, rollout/replicaset.go, rollout/canary.go). Accessed: 2026-11-03.

[18] GitHub user report. Stale 'scale-down-deadline' on an orphaned ReplicaSet causes a reconcile never-ending loop and high CPU usage. https://github.com/argoproj/argo-rollouts/issues/4971, 2026. Argo Rollouts issue #4971, opened 2026-08-11; describes a stale scale-down-deadline annotation suppressing the progress deadline. Accessed: 2026-11-03.

[19] GitHub user contribution. fix(controller): expire stale scale-down-deadline to stop reconcile loop. https://github.com/argoproj/argo-rollouts/pull/4972, 2026. Argo Rollouts pull request #4972, unmerged at the time of access; proposes ignoring expired scale-down deadlines. Accessed: 2026-11-03.

[20] Spinnaker Authors. Kayenta: Automated canary service. https://github.com/spinnaker/kayenta, 2026. Kayenta, an automated canary analysis service using statistical comparison of canary and baseline. Accessed: 2026-11-03.

[21] Gábor Jeney. Hogyan néz ki egy igényes dokumentum? Néhány szóban az alapvető tipográfiai szabályokról, 2014. http://www.mcl.hu/~jeneyg/kinezet.pdf.

[22] Ferenc Wettl, Gyula Mayer, and Péter Szabó. *LaTeX kézikönyv*. Panem Könyvkiadó, 2004.

[23] James C. Candy. Decimation for sigma delta modulation. 34(1):72–76, 01 1986. DOI: 10.1109/TCOM.1986.1096432.

[24] Wai L. Lee and Charles G. Sodini. A topology for higher order interpolative coders. In *Proc. of the IEEE International Symposium on Circuits and Systems*, pages 459–462, 05 4–7 1987.

[25] Peter Kiss. Adaptive digital compensation of analog circuit imperfections for cascaded delta-sigma analog-to-digital converters, 04 2000.

[26] Richard Schreier. *The Delta-Sigma Toolbox v5.2*. Oregon State University, 01 2000. http://www.mathworks.com/matlabcentral/fileexchange/.

[27] Alexey Mkrtychev. Models for the logic of proofs. In Sergei Adian and Anil Nerode, editors, *Logical Foundations of Computer Science*, volume 1234 of *Lecture Notes in Computer Science*, pages 266–275. Springer Berlin Heidelberg, 1997. ISBN 978-3-540-63045-6. DOI: 10.1007/3-540-63045-7_27. URL http://dx.doi.org/10.1007/3-540-63045-7_27.

[28] Budapesti Műszaki és Gazdaságtudományi Egyetem Villamosmérnöki és Informatikai Kar. Diplomaterv portál (2011. február 26.). http://diplomaterv.vik.bme.hu/.

Appendix

A.1 Declaration on the Use of Generative Artificial Intelligence

☐ **I have not used** any generative AI tools.

☐ **I have used generative AI tools.** I have verified the content generated by AI, ensured the accuracy of the outputs, and properly indicated each instance of use in the table below.

Usage type	**Name of Generative AI Tool(s)**	**Affected Sections** (chapter, page number, reference)	**Estimate Proportion of Use** (per usage type)
Literature Review			
Brief Summary of the Prompt			
Program Code Generation			
Brief Summary of the Prompt			
Generating New Ideas or Solution Proposals			
Brief Summary of the Prompt			
Creating an Outline (text structure, bullet points)			
Brief Summary of the Prompt			
Creating Text Blocks			
Brief Summary of the Prompt			
Generating Images for Illustrative Purposes			
Brief Summary of the Prompt			
Data Visualization, Generating Charts Based on Data Points			
Brief Summary of the Prompt			
Preparing a Presentation			
Brief Summary of the Prompt			
Other (please specify)			
Brief Summary of the Prompt			
Aggregated Percentage Value (for the core part of the task):			
Brief Textual Justification of the Aggregated Value:			

A.2 A TeXstudio felülete

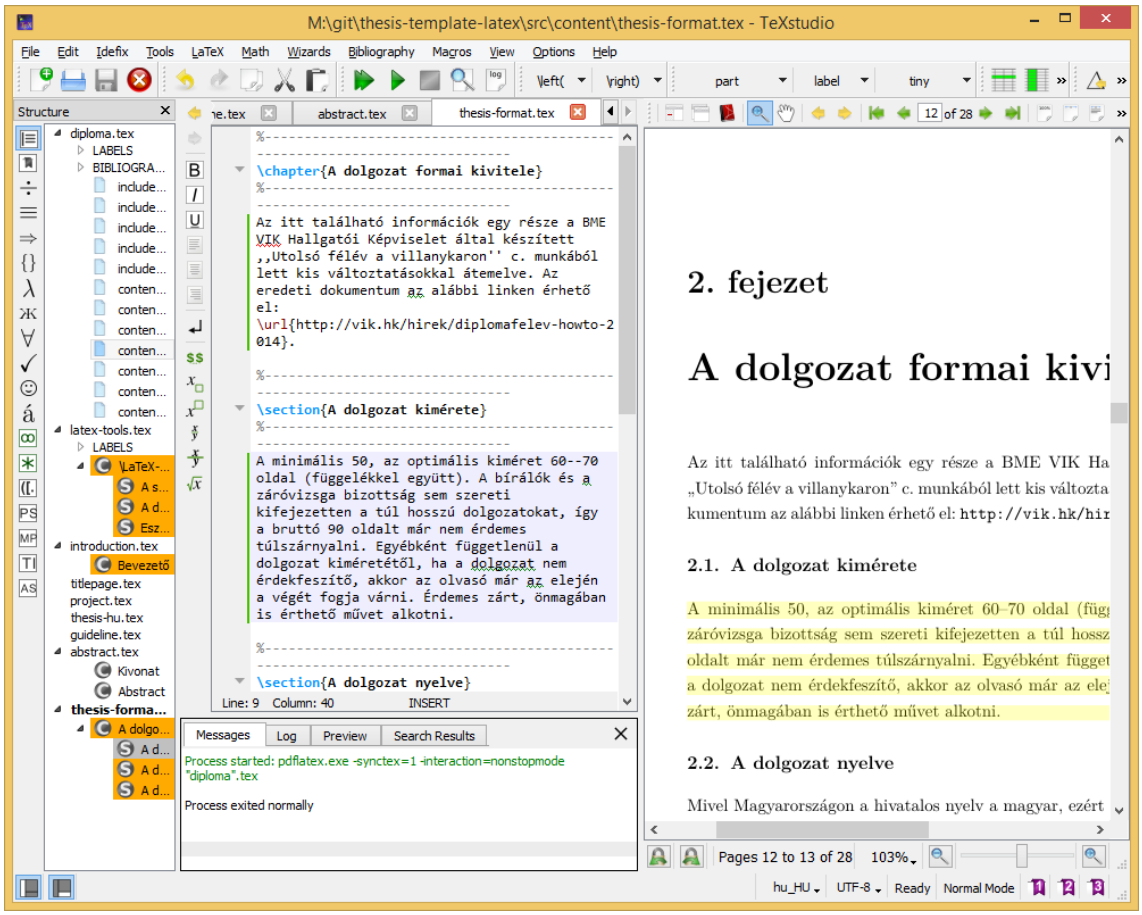


Figure A.2.1: A TeXstudio LaTeX-szerkesztő.

A.3 Válasz az „Élet, a világmindenség, meg minden” kérdésére

A Pitagorasz-tételből levezetve

$$c^2 = a^2 + b^2 = 42. \tag{A.3.1}$$

A Faraday-indukciós törvényből levezetve

$$\operatorname{rot} E = -\frac{dB}{dt} \qquad \longrightarrow \qquad U_i = \oint_{\mathbf{L}} \mathbf{Edl} = -\frac{d}{dt} \int_{A} \mathbf{Bda} = 42. \tag{A.3.2}$$